队伍编号	MCB2303012
赛道	（A）

基于预训练的 DenseNet-121 模型的坑洼道路检测识别

摘 要

本文主要建立了一个预训练的计算机视觉模型，对坑洼道路进行有效的检测。首先对题目给出的数据集进行数据清洗和分析图像特征，再从 CNN 中选取四个较为典型的预训练模型进行分别训练，检测，并选出最优、最适合本题的 **DenseNet-121 模型**进行训练和多维度评估，并最终对题目给出的数据集进行检测。

针对问题一，对训练数据集进行数据清洗，重新分类错误图片，像素统一处理为 **224×224** 大小，之后进行 **7:3** 的训练集和验证集的划分，并提取图像的特征信息。由于题目给出的数据集过少，传统网络模型易出现过拟合现象，会消耗大量计算资源和时间，故本文选用经典的 VGG16、DenseNet-121、GoogLeNet 和 ResNet-50 模型分别训练，对四种模型从多角度分析评估，得到 DenseNet-121 模型的**准确率达到 99.34%，平均迭代时间为 16.02s**，综合评价为四种模型中最佳，**故选用 DenseNet-121 模型进行后续的训练与预测**。在建立 DenseNet-121 模型时，首先引入**随机种子**，加载训练好的参数，对图像进行**数据增强处理**，以提高模型的**泛化能力**；冻结模型的权重，以防止预训练的权重被破坏，并添加一个具有 128 个神经元和 ReLU 激活函数的全连接层进行二分任务，完成 **DenseNet-121 模型**的建立。

针对问题二，对问题一建立的 DenseNet-121 模型进行训练和评估。本文从**准确率、损失函数、训练速度和鲁棒性**等维度进行训练评估。首先测试模型的准确性，将训练好的模型对训练集进行预测，得到了 **99.34%**的准确率，绘制并观察 **ROC 曲线**，得到 **AUC 面积**为 **0.98**，模型性能良好；之后为测试模型的泛化能力，我们下载了新的 681 张图片进行预测，发现对于新的数据集模型仍可达到 **96.34%**的准确率；记录模型的训练耗时信息，得到平均迭代时间 **16.02s**；最后，对图像数据加入扰动，对比其扰动前和扰动后模型的损失率和准确率，发现在 **50 次**的迭代次数下仍可快速的趋于稳定，模型鲁棒性较高，具有识别率高、速度快、分类准确的特点。

针对问题三，将新的数据集同样统一处理为 224×224 的像素大小，在处理中，发现名为 guy7iodk.jpg 的图像无法识别，将其剔除。之后进行特征提取并利用问题二中训练好的模型进行检测，检测过程中生成**随机 16 张**图像与检测结果作为**可视化示例**，将结果存储到 test-result.csv 文件中，通过统计发现共有 **1256** 张图像检测为正常道路，其余 **3685** 张图像为坑洼道路。

关键词：图像分类　CNN　深度学习　DenseNet-121　道路坑洼

目录

一、问题重述.....1
1.1 问题背景.....1
1.2 问题重述.....1
二、问题分析.....1
2.1 问题一的分析.....1
2.2 问题二的分析.....2
2.3 问题三的分析.....2
三、模型假设.....3
四、符号说明.....3
五、问题一模型的建立与求解.....4
5.1 数据预处理.....4
5.1.1 异常值处理.....4
5.1.2 图像归一化.....5
5.1.3 标签编码.....5
5.1.4 数据集划分.....6
5.1.5 数据增强.....6
5.2 CNN 模型的建立.....6
5.2.1 不同神经网络的对比与选择.....7
5.2.2 DenseNet-121 模型的构建.....15
六、问题二模型的建立与求解.....17
6.1 准确率与损失函数.....18
6.2 训练速度评估.....19
6.3 鲁棒性评估.....20
七、问题三模型的建立与求解.....21
7.1 数据预处理.....21
7.2 特征提取.....22
7.3 结果检测.....23
八、模型的评价、改进与推广.....24
8.1 模型的优点.....24
8.2 模型的缺点.....24
8.3 模型的改进.....24
8.4 模型的推广.....24
九、参考文献.....25

一、问题重述

1.1 问题背景

坑洼道路检测和识别是一种计算机视觉任务，旨在通过深度学习技术来识别道路上的坑洼，对于地质勘探、航天科学和自然灾害等领域的研究和应用具有重要意义。传统方法在处理复杂和多变的坑洼图像特征时效果有限，近年来深度学习技术的发展，为坑洼道路检测提供了新的解决方案。深度学习能够提取坑洼的重要特征，并将其转化为更易分类的表示形式。研究者可以使用深度学习方法分类道路图像，也可利用迁移学习来提高性能。要求我们进行分析、特征提取和建模已标记的道路图像，以实现对新道路图像坑洼状态的自动识别。

1.2 问题重述

基于上述问题背景我们需要建立模型解决如下具体问题：

针对问题一，题目要求从提供的图像文件中，使用图像特征提取技术构建一个高准确率、高速度的模型，用于识别道路图像是正常还是有坑洼。

针对问题二，题目要求对构建的模型进行训练，从不同维度进行综合的模型评估。

针对问题三，题目要求利用已经训练好的模型，对测试集中的坑洼图像进行识别，并将识别结果保存在"test_result.csv"文件中。

二、问题分析

2.1 问题一的分析

针对问题一，首先对题目给出的 301 张图像数据集进行了数据清洗和标签化处理，采用 CNN 卷积神经网络中的预训练模型进行训练和测试。针对数据量小的情况，为确保模型具有高学习效率，且避免过拟合，本文选择预训练模型对测试集进行训练，选取了四个代表性的预训练模型，并按照 7:3 的比例划分数据集为训练集和验证集，并对比各个模型的不同指标，最终选择最优的模型进行建立并用于图像识别任务。

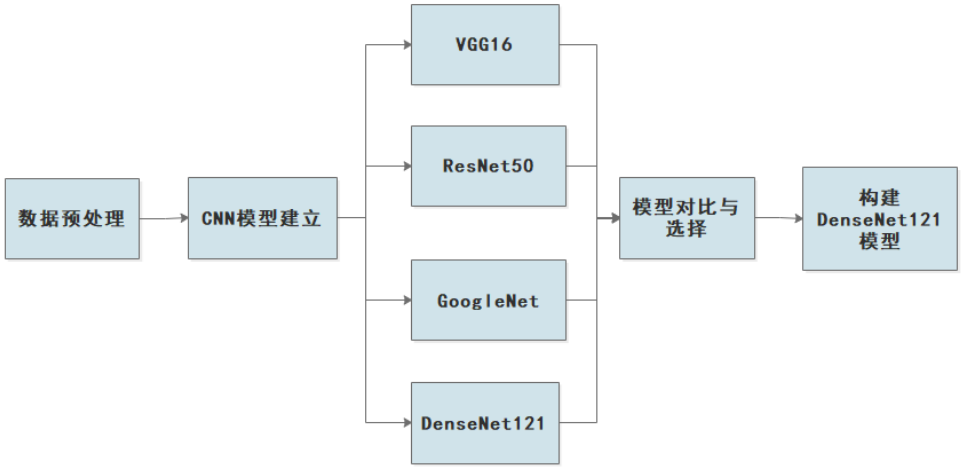


图 2-1 问题一分析流程图

2.2 问题二的分析

针对问题二，本文基于问题一中确定的模型，从不同维度对其进行训练和评估。本文分别从准确率、损失函数、训练速度和鲁棒性评估等方面，通过生成训练集的准确率和损失函数的图像以及对 ROC 曲线和 AUC 面积进行分析和评价，判断模型是否有过拟合现象；引入新数据集，评估模型准确率和泛化能力，计算训练耗时与其他模型对比；对训练数据加入扰动，评估其鲁棒性。

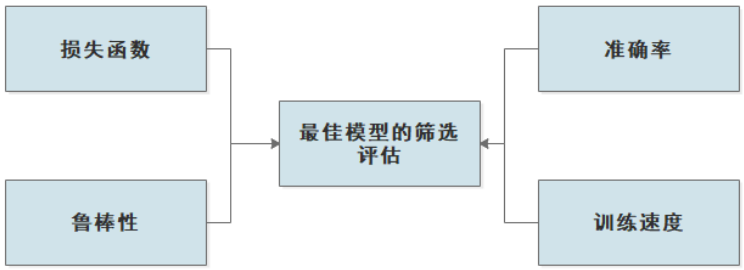


图 2-2 问题二分析流程图

2.3 问题三的分析

针对问题三，本文首先对题目给出的新的数据集观察分析并进行数据预处理，统一像素大小，保证输入的图像符合模型的需求，之后进行图像数据的特征提取并同问题一，问题二的步骤，对新的数据集进行检测，将检测结果保存在 test-result.csv 文件中，对检测结果进行可视化展示，展示不同道路类型的数量和占比。

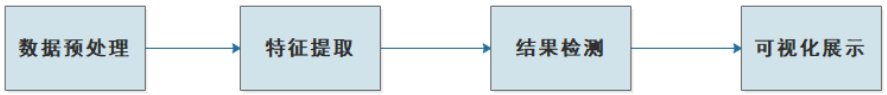


图 2-3 问题三分析流程图

三、模型假设

为确保构建更为精确的模型，本文依据实际情况对模型做出如下合理的假设：

（1）假设训练数据集中提供的图像数据均是真实有效的。

（2）假设图像中轮廓、纹理、形态等坑洼特征可以通过合适的特征提取方法进行有效捕获。

（3）假设模型在训练过程中，运行环境始终保持稳定。

（4）假设模型的训练结果不受主机显卡等硬件因素的影响。

四、符号说明

符号	说明
x_l	第l层的输入
y_l	第l层的输出
W_l	卷积核
b_l	偏置
$f(\cdot)$	激活函数
k	每层增长率
$p(\cdot)$	平均池化操作
C_i	过渡层输入的通道数
$C_i^{(T)}$	过渡层输出的通道数

五、问题一模型的建立与求解

5.1 数据预处理

针对 data.zip 中包含的 301 张图片，我们发现其存在命名不规范、分类错误、像素不一致等问题，因此我们通过 Python 首先将图片命名规范化，正常道路图像统一命名为 $\text{normal}i(i = 1,2,3\cdots)$，坑洼道路统一命名为$\text{potholes}i(i = 1,2,3\cdots)$。随后本文对图像进行异常值处理、图像归一化、标签编码、数据集划分、数据增强等操作，具体流程如图 5-1 所示。

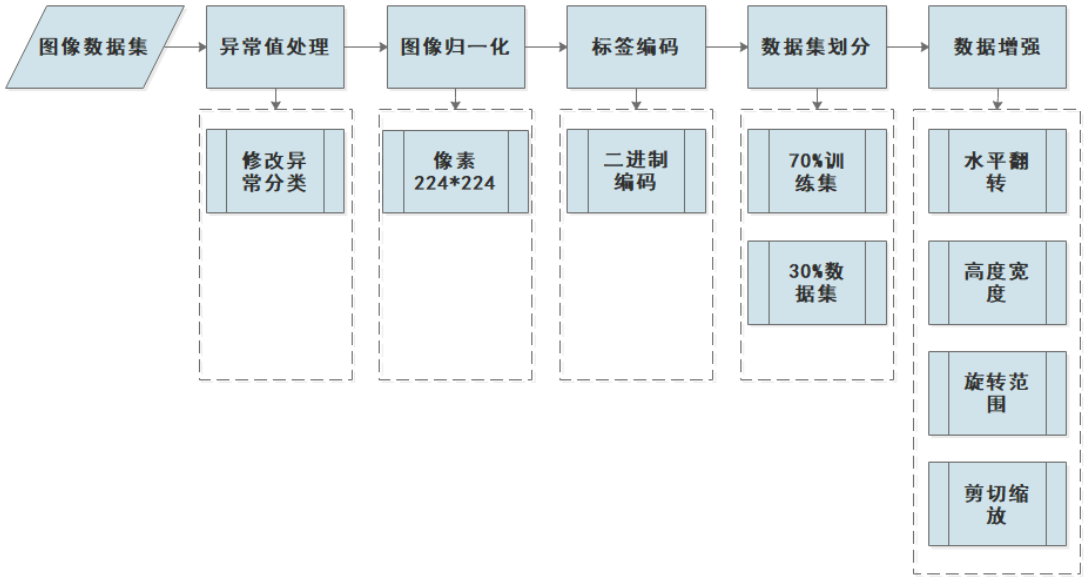


图 5-1 数据预处理流程图

5.1.1 异常值处理

题目所给的训练数据集中，共包含 301 张图片，其中 255 张为正常道路，36 张为坑洼道路。通过筛选，我们发现共有 9 张图片的类型命名错误，具体如图 5-2 所示。异常值的存在会影响后续训练和评估模型，故我们将错误分类图片转换为其正确的分类。分类完成后，我们共得到 246 张正常道路的图片，45 张坑洼道路的图片。



图 5-2 错误分类图像数据

5.1.2 图像归一化

观察附件，我们发现所给数据集中图片的像素大小并不统一，而在深度学习中，模型通常要求输入数据具有固定一致的尺寸和形状，当图像具有不同的大小时，模型可能需要更多的训练迭代才能学习到有效的特征表示。为保留充足图像细节的同时降低计算复杂性，我们通过 Python 将所有的数据集图像调整为相同的尺寸：224x224 像素。

此外，通过调整 rescale 参数，缩放图像的像素值，具体公式如下：

$$x' = \frac{x}{255} \tag{1}$$

其中x表示原始像素值，x'表示归一化后的像素值。

通过归一化处理，我们将像素值从 0 到 255 的范围缩放到 0 到 1 之间，确保模型在训练期间加快收敛速度且具有良好的鲁棒性。

5.1.3 标签编码

标签编码用于将图像标签转化为计算机易于理解的数值形式，以便模型进行有效的学习和预测。由于题目为二元分类问题，即图像要么属于正常道路，要么属于坑洼道路。为确保模型能够识别这两种类别，本文采用二进制标签编码。

二进制标签编码基于二进制数值来表示两个类别之一。通常使用两个不同的值来表示两个类别，例如 0 和 1。这种编码方法在机器学习和深度学习中应用非常广泛，常适用于各种分类算法和模型。

$$y' = \begin{cases} 1, if\ y = positive\ class \\ 0, if\ y = negative\ class \end{cases} \tag{2}$$

其中，y表示原始标签，y'表示编码后的标签。

我们通过 Python 的 sklearn 库中的 LabelEncoder 进行二进制标签编码，具体如表 5-

1 所示：

表 5-1 图像标签编码对应表

正常道路(normal)	坑洼道路(potholes)
1	0

正常道路标签：我们将正常道路的标签编码为 1。表示所有正常道路的图像都将被标记为 1。

坑洼道路标签：我们将坑洼道路的标签编码为 0。表示所有坑洼道路的图像都将被标记为 0。

5.1.4 数据集划分

为准确评估模型的性能并防止过拟合，我们采用数据集划分的方法，将数据按 7:3 比例分为两个部分：训练集和验证集，即 70%的数据训练模型，余下的 30%用于验证模型的性能。

我们使用 Python 的 Scikit-Learn 库中的 train_test_split 函数来执行数据集划分，将数据集分成训练集和验证集，并严格按照 7:3 的比例划分，并设置 stratify 参数，以确保在划分过程中各个类别的样本保持相对均衡。

5.1.5 数据增强

数据增强是通过对原始图像进行随机变换来增加数据样本多样性的一种技术，用于提高深度学习模型的性能和泛化能力。

我们使用 Python 中 TensorFlow 中的 ImageDataGenerator 类来实现数据增强。以下是数据增强具体参数设置：

- **旋转范围：**最大旋转角度为 20 度，模拟不同拍摄角度的道路图像。
- **宽度位移范围：**最大位移范围为图像宽度的 20%，增加位置多样性。
- **高度位移范围：**最大位移范围为图像高度的 20%，增加位置多样性。
- **剪切范围：**最大剪切范围为 20 度，增加图像的形状多样性。
- **缩放范围：**最大缩放范围为 20%，模拟不同距离和尺度的道路图像。
- **水平翻转：**以 50%的概率对图像进行水平翻转，增加图像对称性的多样性。

参数设置在数据增强过程中引入了随机性，并增加训练数据的多样性，确保后续模型能够更好地适应不同场景下的道路图像，从而提高模型对坑洼道路检测和识别的能力。

5.2 CNN 模型的建立

卷积神经网络（Convolutional Neural Network，CNN）是一种深度学习模型，广泛应

用于计算机视觉领域的图像分类、目标检测、图像分割和生成等任务[1]。CNN 由卷积层、汇聚层和全连接层等组件构成，能够学习和提取图像的特征。CNN 通过层层堆叠，从低级特征到高级特征逐渐抽象出图像的信息。这种层级结构使得 CNN 在处理图像数据时具有强大的表达能力和良好的泛化能力。

5.2.1 不同神经网络的对比与选择

数据集的规模通常是影响机器学习和深度学习任务的模型性能和泛化能力的重要因素之一。由于题目给出的 301 张测试集相对较少，易导致模型在训练过程中出现过拟合，泛化能力下降、学习能力不足等问题。因此，我们采用预训练好的模型进行训练，在卷积神经网络中有多种预训练好的网络模型，例如牛津大学提出的 VGG 模型、微软研究院提出的 ResNet 模型、康奈尔大学的 DenseNet 模型和 Google 团队提出的 GoogLeNet 模型。

首先，为保证实验的可复现性和结果的一致性，在每个模型训练前先加入随机种子 42，之后对图像数据集进行统一处理，将像素大小调整为 224×224。批处理样本数被设置为 32，模型的架构为一个 128 个神经元和 ReLU 激活函数的全连接层，以及一个具有 1 个神经元和 Sigmoid 激活函数的输出层，用于二分类任务。最后，我们选取数据集的 70%作为训练集，30%作为测试集进行训练，并进行 50 次的迭代训练。

下面将对这四种重要的模型进行多维度的比较，以确定最适合题目提供的数据集类型的网络模型。

（1） VGG16 (Visual Geometry Group 16)

VGG16 模型使用了较小的 3x3 卷积核进行特征提取。通过堆叠多个小卷积核，VGG16 能够捕捉不同尺度的特征，并提高模型的表达能力。然而，相对于更大的卷积核，VGG16 的参数数量较多。由于该模型的简单结构和良好的性能，它在图像分类和特征提取任务中被广泛使用。VGG16 在多个图像识别挑战中取得了很好的结果，并成为了深度学习领域的经典模型之一[3]。

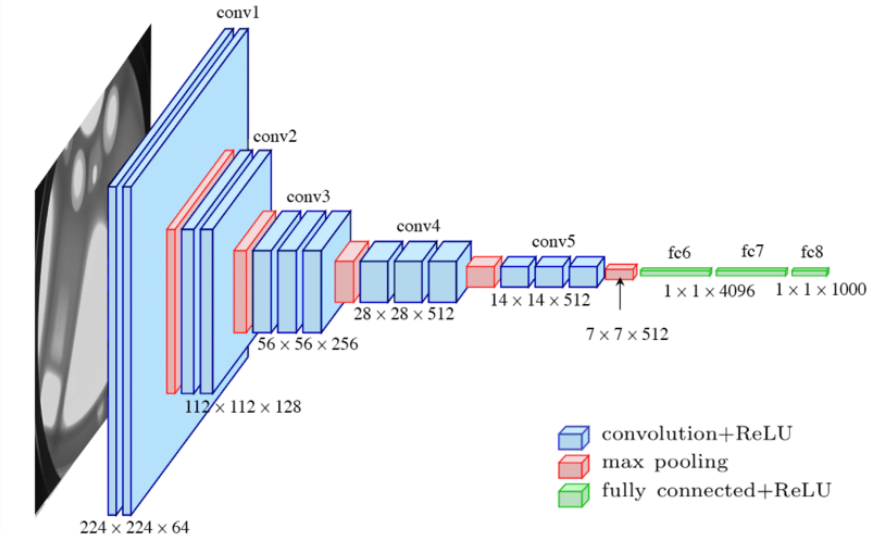


图 5-3 VGG16 模型图

在训练时，我们记录了模型学习过程中的迭代次数和损失率，准确率的详细数值如下表：

表 5-2 VGG16 模型学习数值表

迭代次数	损失率	准确率
1	2.3581	0.7264
2	0.3535	0.8821
3	0.2999	0.8632
4	0.1826	0.9434
5	0.1483	0.9528
6	0.1164	0.9387
7	0.1160	0.9434
8	0.1022	0.9623
9	0.0993	0.9623
10	0.0772	0.9670
...	...	...
40	0.0362	0.9438
41	0.0380	0.9858
42	0.0302	0.9953
43	0.0245	0.9953
44	0.0235	0.9953
45	0.0154	1.0000
46	0.0241	0.9906
47	0.0168	1.0000
48	0.0189	0.9953
49	0.0364	0.9906
50	0.0511	0.9906

通过观察模型训练时产生的详细信息，我们发现该模型的准确率和损失率随迭代次

数增加而逐渐趋于稳定，说明该模型有一定拟合能力，已经学习到了图像的特征信息，并能够在训练集上做出良好的预测。

模型训练完毕后，对测试集进行验证，得到混淆矩阵如下图 5-4 所示：

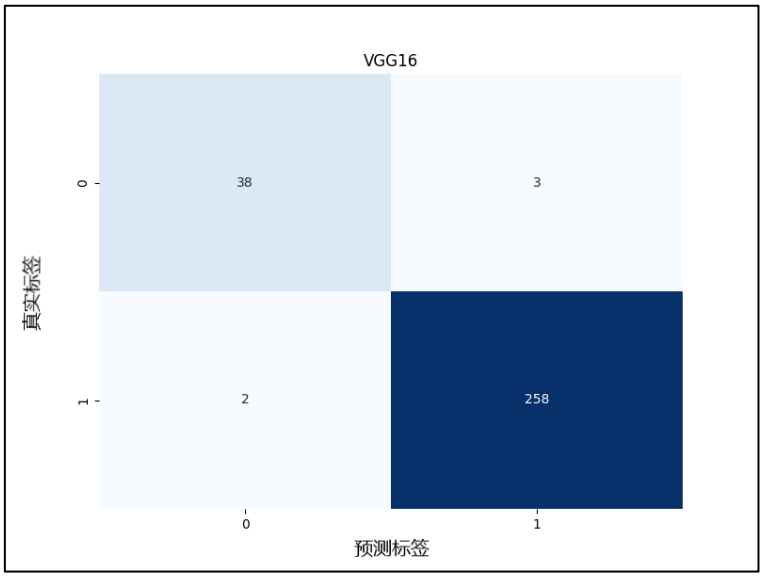


图 5-4 VGG16 混淆矩阵

（2） DenseNet-121 (Densely Connected Convolutional Networks 121)

DenseNet-121 可以构建非常深的网络结构，而不会出现梯度消失的问题。这允许 DenseNet 在更深的层次上学习特征，并且在一些图像分类和目标检测任务中取得了优秀的性能。模型具有更高的参数利用率。由于特征图在层之间的直接连接，特征的重用减少了冗余计算，使得 DenseNet 在参数和计算效率上更加高效[2]。

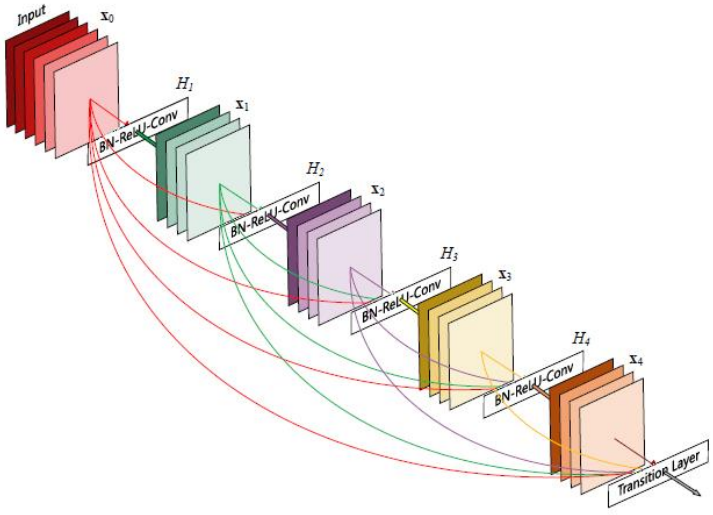


图 5-5 DenseNet121 模型图

模型学习过程中的迭代次数和损失率，准确率的详细数值如下表：

表 5-3 DenseNet-121 模型学习数值表

迭代次数	损失率	准确率
1	0.4416	0.8013
2	0.1505	0.9456
3	0.1170	0.9477
4	0.0684	0.9791
5	0.0567	0.9833
6	0.0603	0.9854
7	0.0439	0.9895
8	0.0425	0.9833
9	0.0452	0.9833
10	0.0462	0.9854
...	...	...
40	0.0168	0.9953
41	0.0297	0.9858
42	0.0121	1.0000
43	0.0174	0.9953
44	0.0320	0.9858
45	0.0094	1.0000
46	0.0099	1.0000
47	0.0157	1.0000
48	0.0081	1.0000
49	0.0105	1.0000
50	0.0185	0.9953

通过观察模型训练时产生的详细信息，我们发现该模型的准确率和损失率随迭代次数增加而逐渐趋于稳定，说明该模型有一定拟合能力。

对测试集进行验证，得到混淆矩阵如下图 5-6 所示：

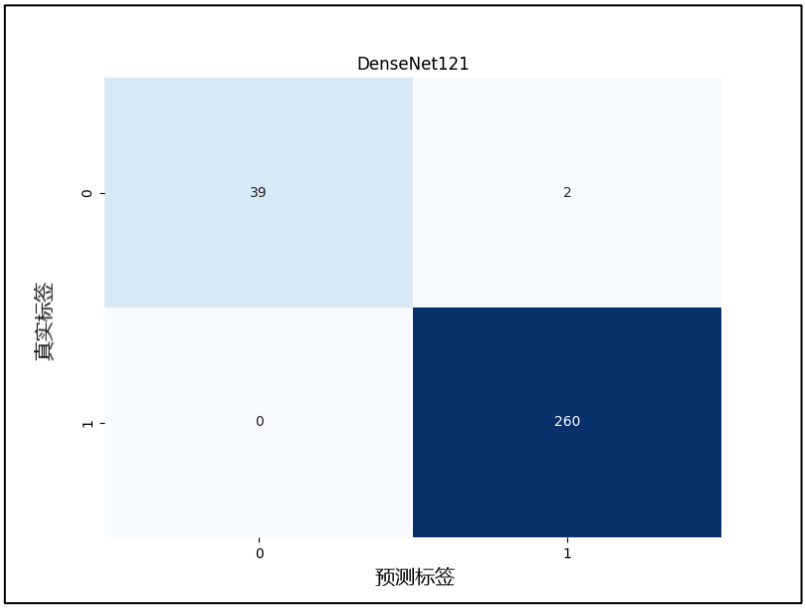


图 5-6 DenseNet-121 混淆矩阵

（3） ResNet-50(Residual Network-50)

ResNet-50 引入了残差连接的概念，通过在网络中添加跳跃连接，将输入直接传递到后续层的输出。这种设计解决了梯度消失问题，允许构建更深的网络。残差连接使得模型更容易训练。ResNet 模型可以构建非常深的网络结构，具有数十甚至上百个卷积层。相比于传统的卷积神经网络，ResNet 的深度结构使得模型能够更好地学习复杂的特征和模式，提高了模型的表达能力[3]。

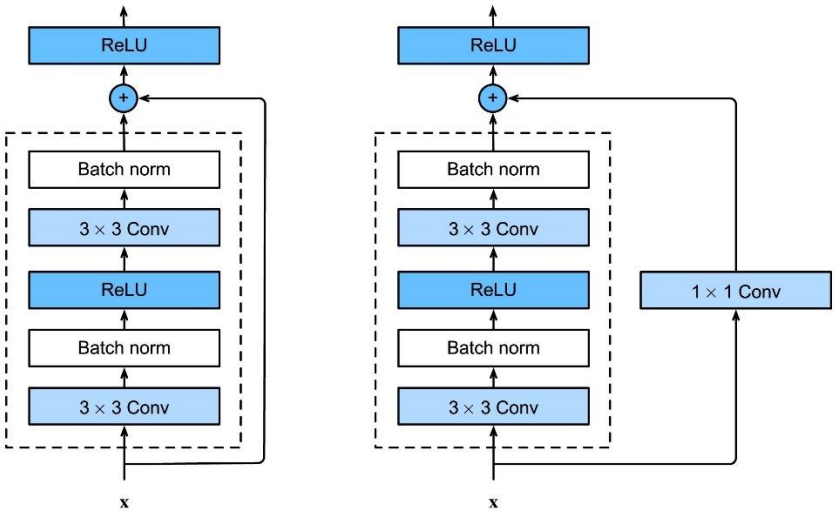


图 5-7 ResNet-50 模型图

模型学习过程中的迭代次数和损失率，准确率的详细数值如下表：

表 5-4 ResNet-50 模型学习数值表

迭代次数	损失率	准确率
1	0.3838	0.8821
2	0.3910	0.8821
3	0.3822	0.8821
4	0.3813	0.8821
5	0.3596	0.8821
6	0.3721	0.8821
7	0.3639	0.8821
8	0.3579	0.8821
9	0.3587	0.8821
10	0.3826	0.8821
...	...	...
40	0.3453	0.8821
41	0.3286	0.8821
42	0.3289	0.8821
43	0.3250	0.8821
44	0.3273	0.8821
45	0.3332	0.8821
46	0.3313	0.8821

47	0.3244	0.8821
48	0.3291	0.8821
49	0.3290	0.8821
50	0.3273	0.8821

通过观察模型训练时产生的详细信息，我们发现该模型的准确率和损失率在训练早期即趋于稳定且保持一个值不变，说明该模型出现欠拟合。该模型在较少的训练集上无法充分学习到数据中的模式和特征，导致准确率较低且不再改善。

对测试集进行验证，得到混淆矩阵如下图 5-8 所示：

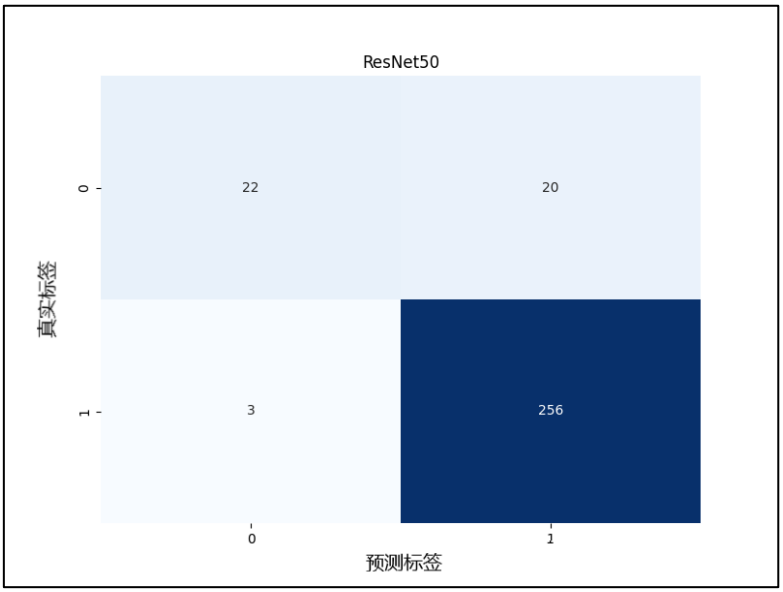


图 5-8 ResNet-50 混淆矩阵

通过观察训练过程中的损失率、准确率以及生成的混淆矩阵等指标，我们注意到在本题目场景下，ResNet-50 模型的性能相对较差，难以很好地对 301 张数据集进行训练和预测。结合该模型本身的结构和适用场景，我们认为 ResNet-50 的网络深度和复杂性可能过高，尤其在训练集较小或无法充分代表整个数据分布的情况下，ResNet-50 可能会在训练过程中过度拟合训练数据，从而导致在未见过的数据上性能下降。

（4） GoogLeNet

GoogLeNet 具有更轻量化的架构，通过使用 Inception 模块和全局平均池化层来减少参数数量和计算复杂度，从而在保持较高的准确性的同时减少了模型的复杂性。为了进一步减少参数数量，GoogLeNet 使用全局平均池化层代替全连接层，将特征图转化为固定大小的向量，有助于减少过拟合，并且减少了需要训练的参数数量。

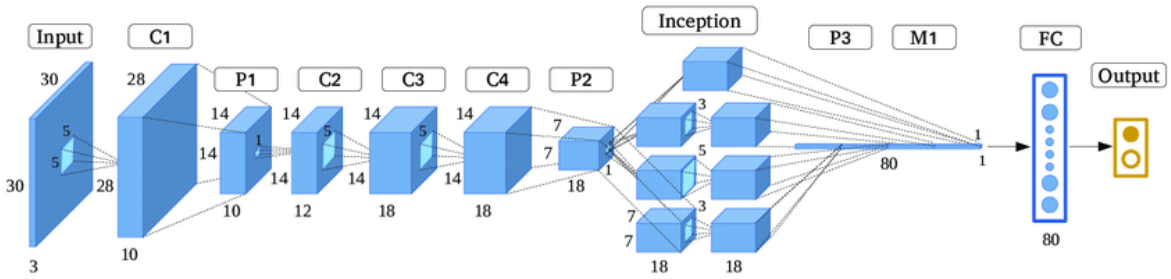


图 5-9 GoogleNet 模型图

模型学习过程中的迭代次数和损失率，准确率的详细数值如下表：

表 5-5 GoogleNet 模型学习数值表

迭代次数	损失率	准确率
1	0.5816	0.8443
2	0.2517	0.9057
3	0.1452	0.9481
4	0.1169	0.9481
5	0.1048	0.9623
6	0.0764	0.9764
7	0.0930	0.9670
8	0.0584	0.9811
9	0.0463	0.9811
10	0.0726	0.9764
...	...	...
40	0.0294	0.9858
41	0.0174	0.9906
42	0.0086	1.0000
43	0.0205	0.9906
44	0.0185	0.9906
45	0.0111	1.0000
46	0.0205	0.9906
47	0.0072	1.0000
48	0.0229	0.9953
49	0.0191	0.9906
50	0.0115	0.9953

通过观察模型训练时产生的详细信息，我们发现该模型的准确率和损失率随迭代次数增加而逐渐趋于稳定，说明该模型有一定拟合能力。

对测试集进行验证，得到混淆矩阵如下图 5-10 所示：

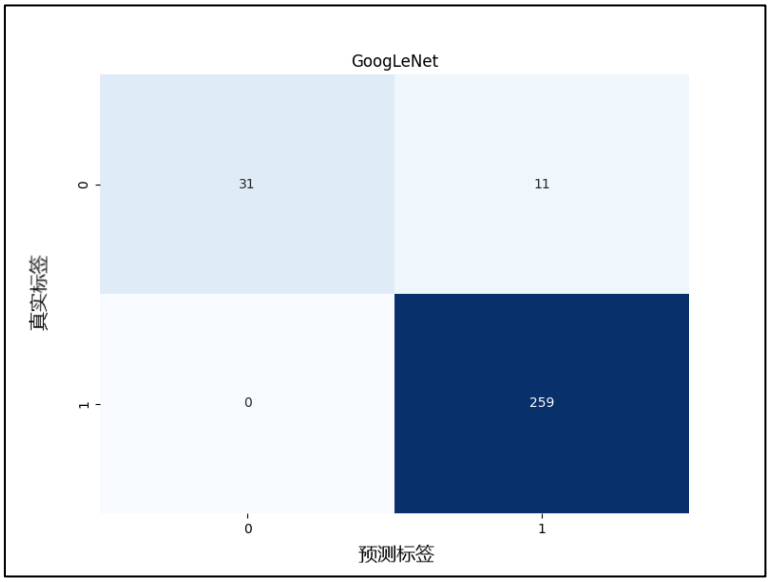


图 5-10 GoogleNet 混淆矩阵

我们将上述四种模型的准确率变化曲线进行对比，如图 5-11 所示，发现 DenseNet-121 模型、VGG16 模型和 GoogLeNet 模型的准确率在 50 次的迭代数下均高且趋于稳定，而 ResNet-50 模型的稳定性较差不作为候选模型继续使用。

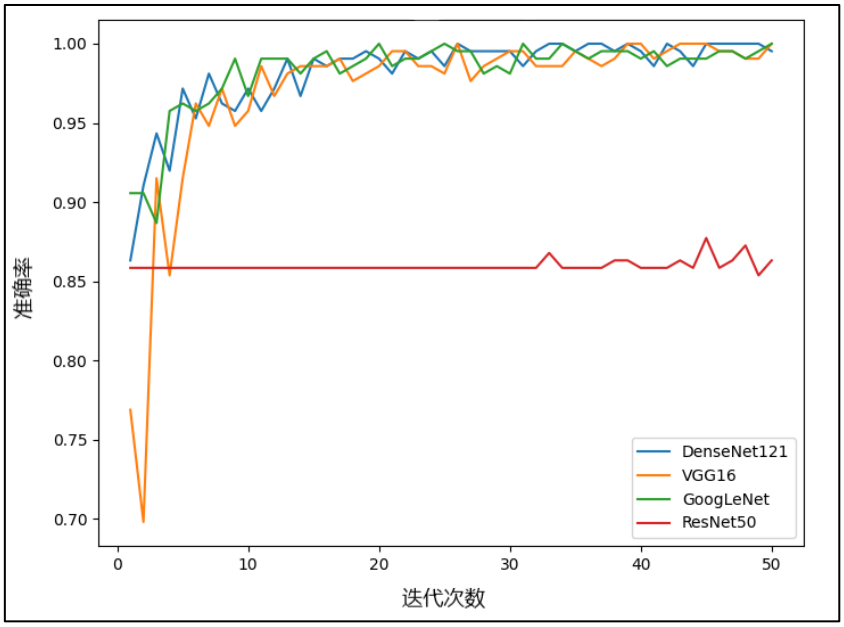


图 5-11 模型准确率随迭代次数变化图

将这四种训练好的模型再次对题目给出 301 张数据集作为测试集，分别预测并将图像分类，得出四种模型的准确率分别为：**DenseNet-121 为 99.34%，VGG16 为 98.34%，GoogLeNet 为 96.35%，而 ResNet-50 的准确率最低，为 92.36%。**

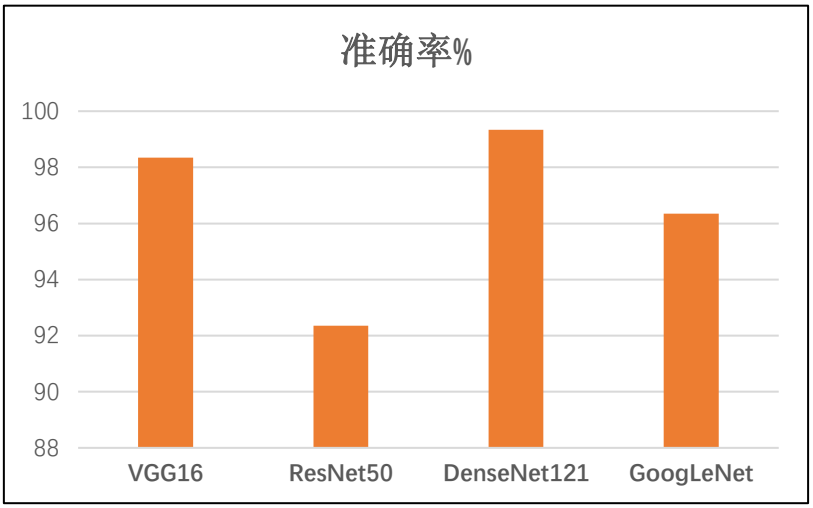


图 5-12 模型准确率对比图

之后，将这四种模型的训练时每次迭代完成的时间进行平均计算，得到模型训练耗时的图像，如图 5-13 所示：

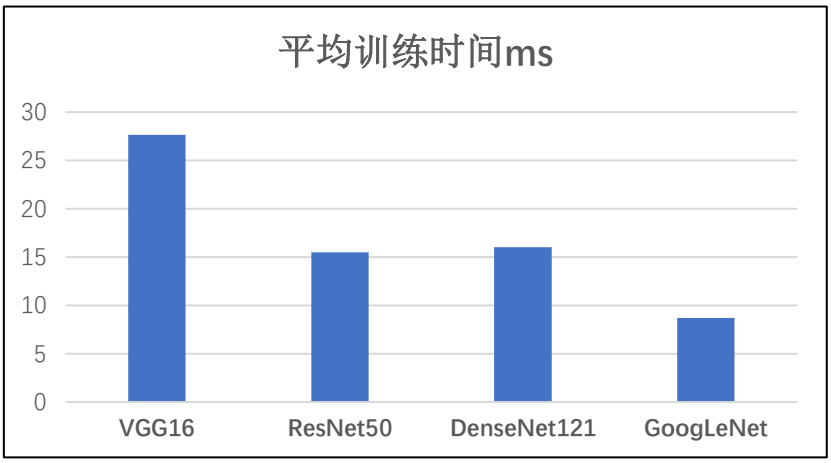


图 5-13 模型平均训练时间

可以看出耗时最少的三个模型为 ResNet-50，DenseNet-121 和 GoogLeNet 模型，但由于 ResNet-50 的稳定性较差，GoogLeNet 模型的准确性低于 DenseNet-121。**综上，本文选择将 DenseNet-121 模型作为最佳模型进行使用。**

5.2.2 DenseNet-121 模型的构建

DenseNet-121 是一种深度卷积神经网络模型，由 121 层（包括卷积层、池化层、全连接层等）组成。该模型的主要特点是密集连接，即在每个层中，将前面所有层的特征图按照通道维度进行拼接，形成新的输入。这种密集连接的方式有助于解决梯度消失问题，能够更好地传递梯度和特征信息，促进了网络的训练和特征的重用。密集连接可以表示为：

$$x_l = H_l([x_0, x_1, \ldots, x_{l-1}]) \tag{3}$$

其中，x_l表示第l层的输出，H_l表示第l层的非线性变换函数（如卷积、批量归一化、激活

函数等），$[x_0, x_1, \ldots, x_{l-1}]$表示将前面所有层的输出按照通道维度进行拼接。

Densenet-121 的基本原理是将网络分为四个密集块和一个过渡层，每个密集块内部的层都是相同的，都是由一个批量归一化、一个激活函数和一个卷积组成[4]。每个过渡层则是由一个批量归一化、一个$1x1$的卷积和一个平均池化组成，主要表示如下：

设第l层的输入为x_l，输出为y_l，卷积核为W_l，偏置为b_l，激活函数为$f(\cdot)$，则有：

$$y_l = f(W_l * x_l + b_l) \tag{4}$$

其中，$*$表示卷积运算。

设第l层之前有$k_0 + (l-1)k$个特征图，其中k_0是初始输入的通道数，k是每层增长率（Growth Rate），则有：

$$x_l = [y_0, y_1, \ldots, y_{l-1}] \tag{5}$$

其中，$[y_0, y_1 \ldots, y_{l-1}]$表示将所有特征图在通道维度上进行拼接。

设第i个过渡层的输入为$x_i^{(T)}$，输出为$y_i^{(T)}$，卷积核为$w_i^{(T)}$，偏置为$b_i^{(T)}$，压缩率（Compression Rate）为$\theta \in (0,1]$，则有：

$$y_i^{(T)} = P(f(W_i^{(T)} * x_i^{(T)} + b_i^{(T)})) \tag{6}$$

其中，$p(\cdot)$表示平均池化操作。过渡层的作用是减少特征图的数量和尺寸，以降低计算量和内存消耗。压缩率θ控制了过渡层输出的特征图数量，即：

$$C_i^{(T)} = \theta C_i \tag{7}$$

其中，$C_i^{(T)}$是过渡层输出的通道数，C_i是过渡层输入的通道数。

本文按照如下步骤建立 DenseNet-121 模型：

Step 1 将训练的图像统一为224×224像素大小，通过调整图像尺寸，确保输入到模型中的图像具有一致的维度，使模型能够更好地学习和提取特征。

Step 2 将训练批次设置为 32，即每个训练步骤中一次性加载到模型中的样本数量。通过较小的批次大小提供更多的随机性和样本多样性，确保模型更好地泛化到未知数据，同时提供更好的梯度估计，助于模型更好地收敛和泛化。

Step 3 通过 Python 使用 keras.applications.DenseNet121 函数创建了一个 DenseNet121 模型 base_model，并通过设置 weights='imagenet’参数，加载了在 ImageNet 数据集上预训练的权重，使得模型能够从先前的大规模数据集中学习到有用的特征；同

时，设置 include_top=False 参数，只保留模型的卷积层部分，将其作为特征提取器；接下来，通过指定 input_shape=(image_height, image_width, 3)参数，定义了输入图像的形状。

Step 4 通过设置 base_model.trainable = False 参数，冻结了 DenseNet121 模型的权重，即在训练过程中不更新这些权重，以防止预训练的权重被破坏。

Step 5 通过 keras.Sequential 创建了一个新的模型 model，其中包含了将 base_model 作为第一层，在后面我们设置跟随了一个全局平均池化层 keras.layers.GlobalAveragePooling2D()，用于将特征图转换为固定长度的特征向量。

Step 6 添加一个具有 128 个神经元和 ReLU 激活函数的全连接层 keras.layers.Dense(128, activation='relu')，用于引入非线性变换并映射特征向量到更高维度的特征空间。

Step 7 添加一个具有 1 个神经元和 Sigmoid 激活函数的全连接层 keras.layers.Dense(1, activation='sigmoid')，用于进行二分类任务，输出输出图像属于正常或者凹坑类别的概率。

基于上述操作，我们建立出 DenseNet-121 神经网络模型，模型具体参数设置如下表所示：

表 5-6 DenseNet-121 模型参数表

参数	参数值	说明
image_height	224	图像高度
image_width	224	图像宽度
batch_size	32	批处理大小
weights	'imagenet'	权重初始化方式
include_top	False	是否包含顶层（全连接层）
input_shape	(224, 224, 3)	输入图像尺寸
base_model	DenseNet-121	基础模型
trainable	False	是否冻结基础模型的权重

DenseNet-121 模型建立完成后，后续本文将基于多维度进行模型的训练和评估。

六、问题二模型的建立与求解

基于建立的 DenseNet-121 模型，我们已经使用训练数据集的 301 张图像按照 7:3 比

例划分训练集和验证集来完成训练。针对训练完成的模型，多维度的模型评估角度十分必要，便于深入了解模型在不同方面的性能，识别模型的优势和弱点，从而进一步的改进和优化。故本文选择从准确率与损失函数、训练速度和鲁棒性四个方面进行模型的评估[5]。

6.1 准确率与损失函数

为研究 DenseNet-121 模型的泛化能力，我们从 kaggle 平台上下载了名为 pothole-detection-dataset 的数据集，得到 681 张新的道路图片，将训练好的模型对该数据集进行预测。首先对该数据集的所有图片进行标签化处理后对其进行预测，生成的新的预测集的混淆矩阵如下：

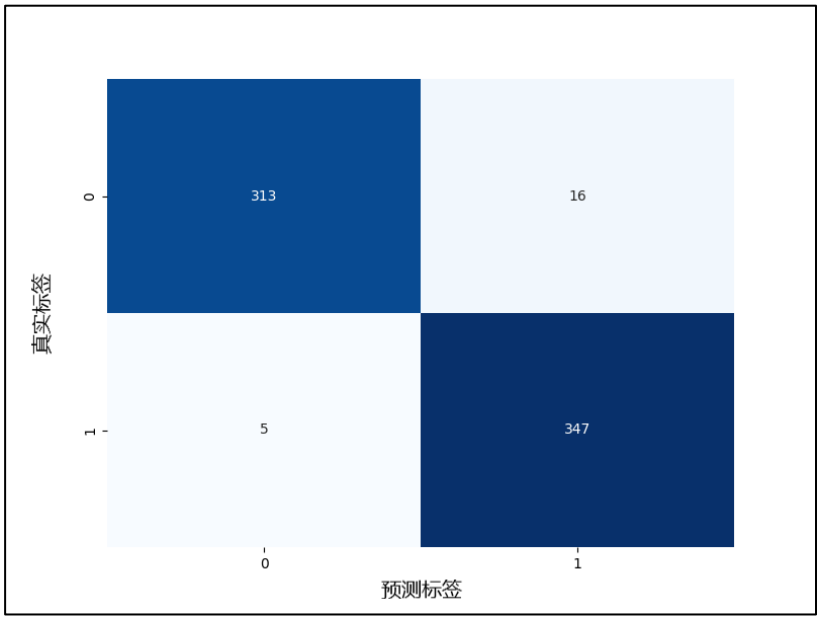


图 6-1 DenseNet-121 模型混淆矩阵

如图 6-1 所示，得到准确率在 96.92%，我们初步判断模型具有较强的泛化能力。泛化能力是指模型在未见过的数据上的性能表现，也可指模型对于不同数据分布和应用场景的适应能力。模型在新的数据集上表现良好时，说明它能够推广到新的数据样本，并适应新的情况。因此，96.92%的准确率表明模型具有一定的泛化能力，可以有效处理新的数据样本。

同时，为进一步评估模型的性能和鉴别其优劣，我们利用 Python 绘制模型的 ROC 曲线，如图 6-2 所示。

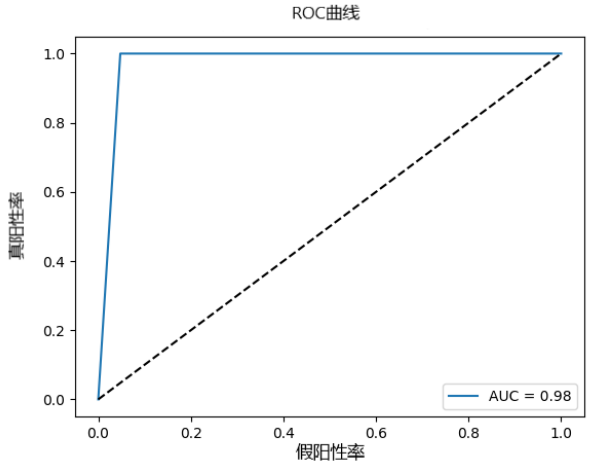


图 6-2 DenseNet-121 模型 ROC 曲线

通过观察 ROC 曲线和 AUC 面积，得到 AUC 面积为 0.98，可以得出 ROC 曲线靠近左上角且 AUC 面积较大，表示模型具有较高的真阳性率和较低的假阳性率，预测性能良好。

将训练过程中的损失曲线和准确率曲线绘制出来进行分析，如图 6-3 所示，可以发现：损失函数随着训练迭代次数的增加而快速减小，并且在训练集和验证集上都呈现下降趋势，说明模型在学习任务上取得了进步；其次，通过比较训练集和验证集的损失函数值，发现模型在训练集上表现优异，在验证集上也表现出较为优秀，模型损失率在第 5 次迭代后稳定在 0.2 之下，说明模型未出现过拟合现象。

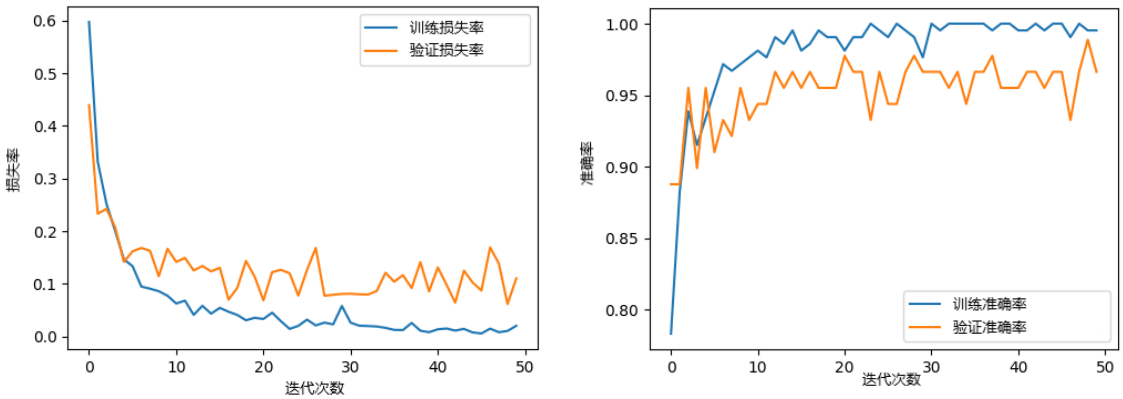


图 6-3 损失曲线和准确率曲线图

6.2 训练速度评估

模型的训练速度和资源消耗是衡量模型效率的重要指标。模型能够在较短的时间内完成训练，并且能够在有限的计算资源下高效运行，意味着模型具有快速的训练速度和较低的资源消耗，在实际应用中的实时性能和成本效益都非常重要。

记录 DenseNet-121 模型在训练过程中的耗时信息，如下表所示：

表 6-1 DenseNet-121 耗时信息表

迭代次数	总时间 s	批次时间 s
1	15	2
2	15	2
3	14	2
4	14	2
5	16	2
6	16	2
7	18	2
8	19	2
9	18	2
10	15	2
...	...	...
40	14	2
41	14	2
42	14	2
43	15	2
44	16	2
45	16	2
46	15	2
47	16	2
48	14	2
49	18	2
50	17	2

通过计算，我们得到平均训练时间 16.02s，该数据在 301 张图片的训练集中的耗时是相对可观的，在同一训练集训练过程中，GoogLeNet 模型的训练时间为 8.69s 每轮迭代，VGG16 模型的训练时间更是达到了 27.65s 每轮迭代。

6.3 鲁棒性评估

模型的鲁棒性是指模型对于输入数据的变化和干扰的稳定性和健壮性，一个鲁棒性较好的模型能够在面对不同类型的输入数据或噪声时保持良好的性能。

本文在训练模型时，将训练集进行一定的扰动，将输入图像进行旋转、缩放、裁剪、添加噪声等操作，并观察模型的输出是否仍然能够正确地进行分类或预测，具体扰动参数设置如下表：

表 6-1 扰动信息表

参数	参数值	说明
rotation_range	20	随机旋转图像的角度

width_shift_range	0.2	随机水平平移图像的宽度范围
height_shift_range	0.2	随机垂直平移图像的高度范围
shear_range	0.2	随机剪切变换图像的错切强度
zoom_range	0.2	随机缩放图像的尺度范围
horizontal_flip	True	随机以 50%的概率水平翻转图像

通过将未进行扰动的模型和进行扰动处理后的模型的准确率和损失率进行对比，如图 6-4 所示，可以发现扰动后的模型准确率依然处于较高的水平，损失率仍然随着迭代次数的增加而快速下降，并趋于稳定。

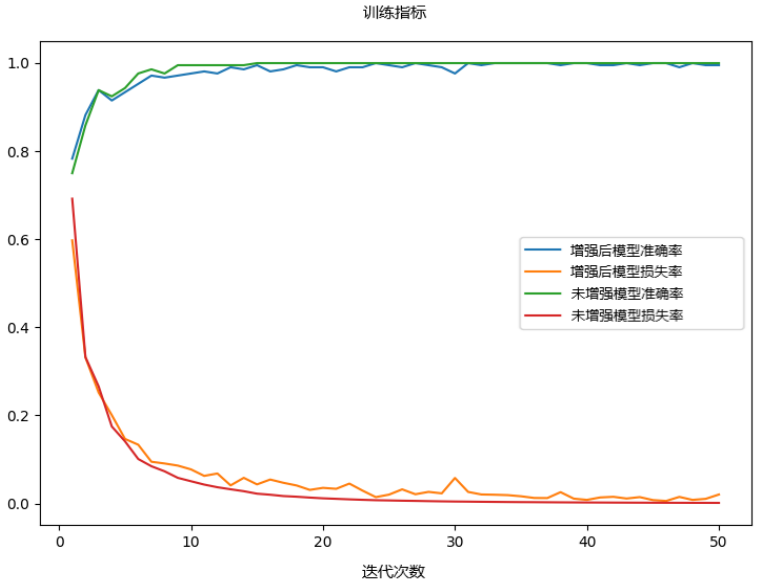


图 6-4 损失曲线和准确率曲线对比图

最后，本文对模型进行跨数据集测试，将模型应用于与训练数据集不同的测试数据集，并观察模型在新数据集上的表现。如果模型在新的数据集上仍然能够保持准确性，则说明模型具有一定的鲁棒性，能够泛化到不同的数据分布。本文在 6.1 准确率与损失函数一节中已经验证，该模型预测新的数据集的准确率可达到 96.92%，因此可以说明模型具有较强的鲁棒性。综上，我们可以认为 DenseNet-121 模型具有识别率高、速度快、分类准确的特点。

七、问题三模型的建立与求解

7.1 数据预处理

同上述处理步骤一致，首先将题目中给出的 4942 张预测集的图像像素统一处理为 224×224 像素大小，具体如表 7-1 所示:

表 7-1 图像大小处理过程表

图像大小处理过程
Resized skc1mm49.jpg to (224, 224)
Resized skdg8f20.jpg to (224, 224)
Resized skk9bite.jpg to (224, 224)
Resized skpmsg2e.jpg to (224, 224)
Resized skrhr1ui.jpg to (224, 224)
Resized sm81v5fs.jpg to (224, 224)
Resized slvybxux.jpg to (224, 224)
Resized slifadb6.jpg to (224, 224)
Resized sn4arphf.jpg to (224, 224)
Resized sm4qli26.jpg to (224, 224)
.....
Resized zzvth22q.jpg to (224, 224)
Resized zxfhwbki.jpg to (224, 224)
Resized zxgm3ar3.jpg to (224, 224)
Resized zy7m9c8x.jpg to (224, 224)
Resized zx9exkdw.jpg to (224, 224)
Resized zynixpux.jpg to (224, 224)
Resized zyylqfs0.jpg to (224, 224)
Resized zzhe6767.jpg to (224, 224)
Resized zzipq9zn.jpg to (224, 224)
Resized zzvom0ax.jpg to (224, 224)

在对测试集进行扫描时，发现名为 guy7iodk.jpg 的图像出现了编码错误，故将其直接删除。其余图像正常处理并保存。

7.2 特征提取

在坑洼道路检测中，特征提取可以将坑洼图像中的轮廓、纹理、形态等关键信息提取出来，便于模型能够更好地识别和分类坑洼，提高任务的准确性和效率。

为了提取图像的特征，我们使用 Python 中的 base_model.predict 方法，该方法基于经过训练的 DenseNet-121 模型的中间层来提取图像的特征表示。这些特征表示可以用于后续的坑洼道路检测，对测试集的特征提取部分数据展示如图 7-1 所示：

```
Image 0i4iakfv:
[[3.7783108e-04 1.1336444e-02 0.0000000e+00 ... 0.0000000e+00
  6.9270271e-01 0.0000000e+00]
 [5.1893090e-04 7.5149741e-03 8.2925865e-03 ... 0.0000000e+00
  1.8814019e+00 0.0000000e+00]
 [5.8725930e-04 3.8658448e-03 3.6771819e-03 ... 2.5751984e-01
  1.2876033e+00 0.0000000e+00]
 ...
 [1.5408118e-04 3.2461833e-03 5.8066002e-03 ... 1.9579110e+00
  1.4146117e+00 0.0000000e+00]
 [0.0000000e+00 4.0194290e-03 5.8783153e-03 ... 8.4811115e-01
  9.2620754e-01 7.3574889e-01]
 [0.0000000e+00 1.5279163e-02 0.0000000e+00 ... 2.0853657e-01
  0.0000000e+00 1.0469673e+00]] [[3.5003162e-04 1.3811637e-02 9.2626549e-04 ... 5.7437718e-02
  9.3266106e-01 0.0000000e+00]
 [7.3807599e-04 6.1230012e-03 3.8638809e-03 ... 0.0000000e+00
  2.2795219e+00 0.0000000e+00]
 [0.0000000e+00 4.0757458e-04 1.3345449e-02 ... 1.1137798e+00
  1.7332528e+00 0.0000000e+00]
 ...
 [5.5830495e-04 0.0000000e+00 0.0000000e+00 ... 2.8080637e+00
  2.4419765e+00 5.0000846e-02]
 [8.5575302e-05 2.8711883e-04 0.0000000e+00 ... 2.3083036e+00
  1.8685530e+00 1.4007800e+00]
 [2.0690559e-04 1.6849682e-02 0.0000000e+00 ... 1.4015732e+00
  6.9212073e-01 1.3540261e+00]] [[2.46354961e-04 3.15766595e-03 0.00000000e+00 ... 0.00000000e+00
  0.00000000e+00 6.39930964e-01]
 [5.18825720e-04 0.00000000e+00 4.70432406e-03 ... 4.83730435e-02
  6.51570916e-01 6.45007491e-02]
 [9.67239612e-04 0.00000000e+00 1.24328155e-02 ... 2.41250849e+00
  1.51153874e+00 0.00000000e+00]]
```

图 7-1 特征提取图

7.3 结果检测

我们使用 DenseNet-121 模型对测试集进行测试，为更清楚地展示道路识别模型的性能，在图像检测过程中，我们生成了随机的 16 张图像的检测结果作为可视化示例展示，具体如图 7-2 所示：



图 7-2 随机 16 张检测结果图

通过图 7-2，我们得到在生成的随机 16 张图像的检测结果中，共有 11 张图像为正常道路，5 张图像为坑洼道路，经核对后得出其随机 16 张图像的检测结果全部正确。

所有图像检测完成后，我们将检测结果保存到“test_result.csv”文件中，其中包含图像文件名以及分类标识。通过筛选标签值，我们得到检测结果中共有 1256 张图像为正常道路，共有 3685 张图像为坑洼道路。最后，将检测结果进行可视化展示，如图 7-3 所示。

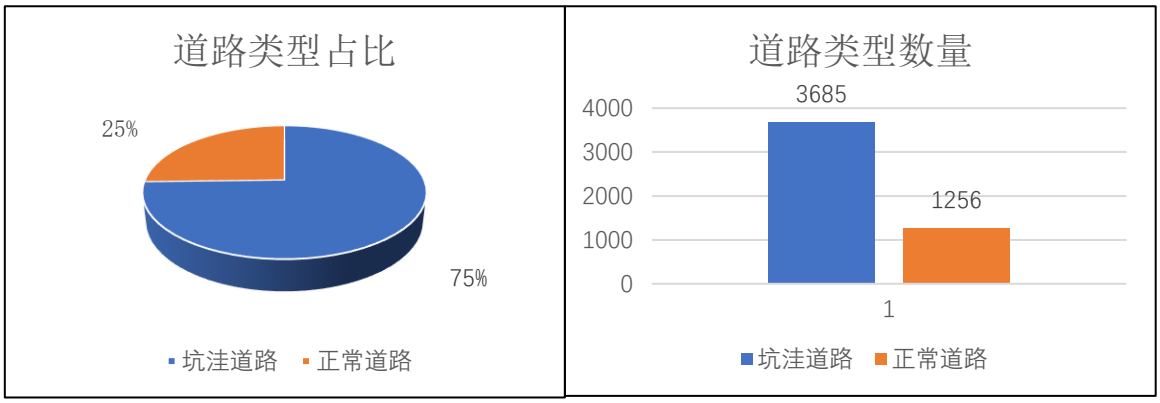


图 7-3 检测结果可视化

八、模型的评价、改进与推广

8.1 模型的优点

（1）本文基于多个角度进行模型对比，得到 DenseNet-121 模型具有出色的性能，能够实现更高的准确率，识别速度和分类准确性，尤其在有限的数据集上

（2）DenseNet-121 模型采用密集连接，每一层都与前面的所有层相连接，有助于梯度传播和特征重用，减少了梯度消失问题。

（3）由于密集连接的特性，DenseNet-121 模型对于小型数据集更具鲁棒性，更不容易过拟合。

8.2 模型的缺点

（1）DenseNet-121 模型的计算和内存需求相对较高，资源有限的环境下运行困难。

（2）由于其深度和连接密集性，且训练数据集较小等因素，DenseNet-121 模型的训练时间可能较长，需要更多的计算资源。

8.3 模型的改进

DenseNet-121 模型在训练过程中可以引入更多图像数据，提升训练效果和准确率，并可以通过分布式训练进一步加速训练过程，以减少训练时间。

8.4 模型的推广

本文主要建立的预训练的 DenseNet-121 模型不仅适用道路坑洼识别等图像分类任务，还可推广到目标检测、图像分割、视频分析等多个计算机视觉领域[6]。且 DenseNet 的思想也可应用于自然语言处理和语音识别等其他领域，以及自动驾驶、医学影像分析等领域，以改进模型性能和泛化能力，对提高机器学习模型的效率和性能具有重要意义。

九、参考文献

[1]周忠义, 吴谨, 朱磊. 基于多路特征融合和深度学习的露霜图像分类[J]. 计算机应用与软件, 2018, 35(10): 205-211.

[2]Huang G, Liu Z, Pleiss G, et al. Convolutional networks with dense connectivity[J]. IEEE transactions on pattern analysis and machine intelligence, 2019, 44(12): 8704-8716.

[3]Theckedath D, Sedamkar R R. Detecting affect states using VGG16, ResNet50 and SE-ResNet50 networks[J]. SN Computer Science, 2020, 1: 1-7.

[4]Asad M H, Khaliq S, Yousaf M H, et al. Pothole Detection Using Deep Learning: A Real-Time and AI-on-the-Edge Perspective[J]. Advances in Civil Engineering, 2022, 2022.

[5]Ma N, Fan J, Wang W, et al. Computer vision for road imaging and pothole detection: a state-of-the-art review of systems and algorithms[J]. Transportation safety and Environment, 2022, 4(4): tdac026.

[6]Iandola F, Moskewicz M, Karayev S, et al. Densenet: Implementing efficient convnet descriptor pyramids[J]. arXiv preprint arXiv:1404.1869, 2014.

附录

软件名称：Pycharm2023.01、Excel2019

附录 1

介绍：该代码是使用 python 语言编写，是 DenseNet-121 模型的代码

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
import h5py

np.random.seed(42)
tf.random.set_seed(42)

data_dir = 'E:\Acode\AAAA\\test_v2.0'

image_height = 224
image_width = 224
batch_size = 16

# 创建 DenseNet121 模型
base_model = keras.applications.DenseNet121(
    weights='imagenet',
    include_top=False,
    input_shape=(image_height, image_width, 3)
)

# 冻结 DenseNet121 模型的权重
base_model.trainable = False

# 创建新的模型架构
model = keras.Sequential([
    base_model,
    keras.layers.GlobalAveragePooling2D(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.summary()
```

```
# 数据集划分为训练集和验证集
train_data = ImageDataGenerator(
    rescale=1. / 255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=0.3
)

train_generator = train_data.flow_from_directory(
    data_dir,
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='binary',
    shuffle=True,
    subset='training',
    classes=['normal', 'potholes']
)

validation_generator = train_data.flow_from_directory(
    data_dir,
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='binary',
    shuffle=True,
    subset='validation',
    classes=['normal', 'potholes']
)

epochs = 50

history = model.fit(
    train_generator,
    steps_per_epoch=len(train_generator),
    epochs=epochs,
    validation_data=validation_generator,
    validation_steps=len(validation_generator)
)

h5_file = 'E:\Acode\AAAA\Lineh5\DenseNet121.h5'

with h5py.File(h5_file, 'w') as f:
    for key, value in history.history.items():
```

```
        f.create_dataset(key, data=value)


def plot_model_performance(history):
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()


plot_model_performance(history)

# 保存训练好的模型
model.save('E:\Acode\AAAA\\trained_model_DenseNet121.h5')

print("训练好的模型保存完毕。")
```

附录 2

介绍：该代码是使用 python 语言编写，是 ResNet-50 模型的代码

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
import h5py

np.random.seed(42)
tf.random.set_seed(42)

data_dir = 'E:\Acode\AAAA\\test_v2.0'

image_height = 224
image_width = 224
batch_size = 32
```

```
# 创建 ResNet50 模型
base_model = keras.applications.ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(image_height, image_width, 3)
)

# 冻结 ResNet50 模型的权重
base_model.trainable = False

# 创建新的模型架构
model = keras.Sequential([
    base_model,
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.summary()

# 数据集划分为训练集和测试集
train_data = ImageDataGenerator(
    rescale=1. / 255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=0.3  # 指定验证集的比例
)

train_generator = train_data.flow_from_directory(
    data_dir,
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='binary',
    shuffle=True,
    subset='training',  # 使用训练集的子集
    classes=['normal', 'potholes']  # 指定类别名称
)
```

```
validation_generator = train_data.flow_from_directory(
    data_dir,
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='binary',
    shuffle=True,
    subset='validation',  # 使用验证集的子集
    classes=['normal', 'potholes']  # 指定类别名称
)

epochs = 50

history = model.fit(
    train_generator,
    steps_per_epoch=len(train_generator),
    epochs=epochs,
    validation_data=validation_generator,
    validation_steps=len(validation_generator)
)

h5_file = 'E:\Acode\AAAA\Lineh5\ResNet50.h5'

with h5py.File(h5_file, 'w') as f:
    for key, value in history.history.items():
        f.create_dataset(key, data=value)


def plot_model_performance(history):
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'],
             label='Validation Accuracy')  # 添加验证集准确率
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')  # 添加
验证集损失
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()
```

```
plot_model_performance(history)

# 保存训练好的模型
model.save('E:\Acode\AAAA\\trained_model_ResNet50.h5')

print("训练好的模型保存完毕。")
```

附录 3

介绍：该代码是使用 python 语言编写，是 GoogLeNet 模型的代码

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
import h5py

np.random.seed(42)
tf.random.set_seed(42)

data_dir = 'E:\Acode\AAAA\\test_v2.0'

image_height = 224
image_width = 224
batch_size = 32

# 创建 GoogLeNet 模型
base_model = keras.applications.InceptionV3(
    weights='imagenet',
    include_top=False,
    input_shape=(image_height, image_width, 3)
)

# 冻结 GoogLeNet 模型的权重
base_model.trainable = False

# 创建新的模型架构
model = keras.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
```

```
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.summary()

# 数据集划分为训练集和测试集
train_data = ImageDataGenerator(
    rescale=1. / 255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=0.3  # 指定验证集的比例
)

train_generator = train_data.flow_from_directory(
    data_dir,
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='binary',
    shuffle=True,
    subset='training',  # 使用训练集的子集
    classes=['normal', 'potholes']  # 指定类别名称
)

validation_generator = train_data.flow_from_directory(
    data_dir,
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='binary',
    shuffle=True,
    subset='validation',  # 使用验证集的子集
    classes=['normal', 'potholes']  # 指定类别名称
)

epochs = 50

history = model.fit(
    train_generator,
    steps_per_epoch=len(train_generator),
    epochs=epochs,
    validation_data=validation_generator,
    validation_steps=len(validation_generator)
)
```

```
h5_file = 'E:\Acode\AAAA\Lineh5\GoogLeNet.h5'

with h5py.File(h5_file, 'w') as f:
    for key, value in history.history.items():
        f.create_dataset(key, data=value)


def plot_model_performance(history):
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'],
             label='Validation Accuracy')  # 添加验证集准确率
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')  # 添加
验证集损失
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()


plot_model_performance(history)

# 保存训练好的模型
model.save('E:\Acode\AAAA\\trained_model_GoogLeNet.h5')

print("训练好的模型保存完毕。")
```

附录 4

介绍：该代码是使用 python 语言编写，是 VGG16 模型的代码

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
import h5py

np.random.seed(42)
tf.random.set_seed(42)
```

```
data_dir = 'E:\Acode\AAAA\\test_v2.0'

image_height = 224
image_width = 224
batch_size = 32

# 创建 VGG16 模型
base_model = keras.applications.VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(image_height, image_width, 3)
)

# 冻结 VGG16 模型的权重
base_model.trainable = False

# 创建新的模型架构
model = keras.Sequential([
    base_model,
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.summary()

# 数据集划分为训练集和测试集
train_data = ImageDataGenerator(
    rescale=1. / 255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=0.3  # 指定验证集的比例
)

train_generator = train_data.flow_from_directory(
    data_dir,
    target_size=(image_height, image_width),
```

```
    batch_size=batch_size,
    class_mode='binary',
    shuffle=True,
    subset='training',  # 使用训练集的子集
    classes=['normal', 'potholes']  # 指定类别名称
)

validation_generator = train_data.flow_from_directory(
    data_dir,
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='binary',
    shuffle=True,
    subset='validation',  # 使用验证集的子集
    classes=['normal', 'potholes']  # 指定类别名称
)

epochs = 50

history = model.fit(
    train_generator,
    steps_per_epoch=len(train_generator),
    epochs=epochs,
    validation_data=validation_generator,
    validation_steps=len(validation_generator)
)

h5_file = 'E:\Acode\AAAA\Lineh5\VGG16.h5'

with h5py.File(h5_file, 'w') as f:
    for key, value in history.history.items():
        f.create_dataset(key, data=value)


def plot_model_performance(history):
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'],
             label='Validation Accuracy')  # 添加验证集准确率
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Train Loss')
```

```
    plt.plot(history.history['val_loss'], label='Validation Loss')  # 添加
验证集损失
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()


plot_model_performance(history)

# 保存训练好的模型
model.save('E:\Acode\AAAA\\trained_model_VGG16.h5')

print("训练好的模型保存完毕。")
```

附录 5

介绍：该代码是使用 python 语言编写，是生成混淆矩阵的代码

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# 从 CSV 文件中读取真实值和预测值
data = pd.read_csv('ans.csv')

y_true = data['True Labels']
y_pred = data['Predicted Labels']

# 生成混淆矩阵
cm = confusion_matrix(y_true, y_pred)

# 可视化混淆矩阵
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, cmap="Blues", fmt="d", cbar=False)
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('ResNet50')  # 添加图片名称
plt.show()
```

附录 6

介绍：该代码是使用 python 语言编写，是修改图像名称的代码

```
import os

# 文件夹路径
```

```
folder_path = "E:\Acode\\testdata_V2"

# 获取文件夹中的所有图像文件
image_files = [f for f in os.listdir(folder_path) if f.endswith(
    ('.jpg', '.jpeg', '.png', '.gif'))]

# 对图像文件进行排序
image_files.sort()

# 重命名图像文件
for i, filename in enumerate(image_files, start=1):
    # 构建新的文件名
    new_filename = str(i) + os.path.splitext(filename)[1]

    # 构建旧的文件路径和新的文件路径
    old_filepath = os.path.join(folder_path, filename)
    new_filepath = os.path.join(folder_path, new_filename)

    # 重命名文件
    os.rename(old_filepath, new_filepath)
```

附录 7

介绍：该代码是使用 python 语言编写，是生成图像损失率和准确率的代码

```
import h5py
import matplotlib.pyplot as plt

h5_files = ['E:\Acode\AAAA\Lineh5\DenseNet121-32.h5',
            'E:\Acode\AAAA\Lineh5\DenseNet121-q.h5']
curve_labels = ['Model 1', 'Model 2']

plt.figure(figsize=(12, 6))

for h5_file, label in zip(h5_files, curve_labels):
    with h5py.File(h5_file, 'r') as f:
        accuracy = f['accuracy'][:]
        loss = f['loss'][:]

    epochs = range(1, len(accuracy) + 1)

    # 绘制准确率曲线
    plt.plot(epochs, accuracy, label='{} - Accuracy'.format(label))

    # 绘制损失率曲线
    plt.plot(epochs, loss, label='{} - Loss'.format(label))
```

```
plt.xlabel('Epochs')
plt.ylabel('Value')
plt.title('Training Metrics')
plt.legend()
plt.show()
```

附录 8

介绍：该代码是使用 python 语言编写，是提取图像特征的代码

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import h5py


np.random.seed(42)
tf.random.set_seed(42)


# 加载图像数据集
data_dir = 'E:\Acode\\testdata_V2'  # 数据集路径
image_height = 224
image_width = 224
batch_size = 32

# 创建 DenseNet121 模型
base_model = keras.applications.DenseNet121(
    weights='imagenet',
    include_top=False,
    input_shape=(image_height, image_width, 3)
)

# 加载并预处理图像数据
data_generator = keras.preprocessing.image.ImageDataGenerator(
    rescale=1. / 255
)

data_flow = data_generator.flow_from_directory(
    data_dir,
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode=None,  # 设置为 None 以仅返回图像数据，不返回标签
    shuffle=False  # 禁止打乱数据
)

# 提取图像的特征
```

```
features = base_model.predict(data_flow, verbose=1)

# 保存特征信息到 txt 文件
output_file = 'image_features.txt'

with open(output_file, 'w') as f:
    for i, feature in enumerate(features):
        feature_str = ' '.join([str(f) for f in features[i]])
        f.write(f'{i+1}:\n{feature_str}\n\n')

print(f"特征信息已保存到文件 {output_file} 中。")
```

附录 9

介绍：该数据是 DenseNet-121 模型在训练时产生的部分信息

```
Epoch 1/50
7/7 [==============================] - 17s 2s/step - loss: 0.3838 - accuracy: 0.8821 - val_loss: 0.3696 - val_accuracy: 0.8876
Epoch 2/50
7/7 [==============================] - 14s 2s/step - loss: 0.3910 - accuracy: 0.8821 - val_loss: 0.3604 - val_accuracy: 0.8876
Epoch 3/50
7/7 [==============================] - 14s 2s/step - loss: 0.3822 - accuracy: 0.8821 - val_loss: 0.3503 - val_accuracy: 0.8876
Epoch 4/50
7/7 [==============================] - 14s 2s/step - loss: 0.3813 - accuracy: 0.8821 - val_loss: 0.3480 - val_accuracy: 0.8876
Epoch 5/50
7/7 [==============================] - 13s 2s/step - loss: 0.3596 - accuracy: 0.8821 - val_loss: 0.3475 - val_accuracy: 0.8876
Epoch 6/50
7/7 [==============================] - 13s 2s/step - loss: 0.3721 - accuracy: 0.8821 - val_loss: 0.3483 - val_accuracy: 0.8876
Epoch 7/50
7/7 [==============================] - 12s 2s/step - loss: 0.3639 - accuracy: 0.8821 - val_loss: 0.3444 - val_accuracy: 0.8876
Epoch 8/50
7/7 [==============================] - 13s 2s/step - loss: 0.3579 - accuracy: 0.8821 - val_loss: 0.3401 - val_accuracy: 0.8876
Epoch 9/50
7/7 [==============================] - 13s 2s/step - loss: 0.3587 - accuracy: 0.8821 - val_loss: 0.3370 - val_accuracy: 0.8876
Epoch 10/50
7/7 [==============================] - 13s 2s/step - loss: 0.3826 - accuracy: 0.8821 - val_loss: 0.3478 - val_accuracy: 0.8876
.....
```

Epoch 40/50
7/7 [==============================] - 13s 2s/step - loss: 0.3453 - accuracy: 0.8821 - val_loss: 0.3142 - val_accuracy: 0.8876
Epoch 41/50
7/7 [==============================] - 13s 2s/step - loss: 0.3286 - accuracy: 0.8821 - val_loss: 0.3069 - val_accuracy: 0.8876
Epoch 42/50
7/7 [==============================] - 13s 2s/step - loss: 0.3289 - accuracy: 0.8821 - val_loss: 0.3109 - val_accuracy: 0.8876
Epoch 43/50
7/7 [==============================] - 13s 2s/step - loss: 0.3250 - accuracy: 0.8821 - val_loss: 0.3132 - val_accuracy: 0.8876
Epoch 44/50
7/7 [==============================] - 13s 2s/step - loss: 0.3273 - accuracy: 0.8821 - val_loss: 0.3153 - val_accuracy: 0.8876
Epoch 45/50
7/7 [==============================] - 13s 2s/step - loss: 0.3332 - accuracy: 0.8821 - val_loss: 0.3230 - val_accuracy: 0.8876
Epoch 46/50
7/7 [==============================] - 13s 2s/step - loss: 0.3313 - accuracy: 0.8821 - val_loss: 0.3129 - val_accuracy: 0.8876
Epoch 47/50
7/7 [==============================] - 14s 2s/step - loss: 0.3244 - accuracy: 0.8821 - val_loss: 0.3161 - val_accuracy: 0.8876
Epoch 48/50
7/7 [==============================] - 13s 2s/step - loss: 0.3291 - accuracy: 0.8821 - val_loss: 0.2991 - val_accuracy: 0.8876
Epoch 49/50
7/7 [==============================] - 13s 2s/step - loss: 0.3290 - accuracy: 0.8821 - val_loss: 0.2972 - val_accuracy: 0.8876
Epoch 50/50
7/7 [==============================] - 13s 2s/step - loss: 0.3273 - accuracy: 0.8821 - val_loss: 0.3031 - val_accuracy: 0.8876

附录 10

介绍：该数据是 ResNet-50 模型在训练时产生的部分信息

Epoch 1/50
7/7 [==============================] - 17s 2s/step - loss: 0.3838 - accuracy: 0.8821 - val_loss: 0.3696 - val_accuracy: 0.8876
Epoch 2/50
7/7 [==============================] - 14s 2s/step - loss: 0.3910 - accuracy: 0.8821 - val_loss: 0.3604 - val_accuracy: 0.8876
Epoch 3/50
7/7 [==============================] - 14s 2s/step - loss: 0.3822 - accuracy: 0.8821 - val_loss: 0.3503 - val_accuracy: 0.8876

```
Epoch 4/50
7/7 [==============================] - 14s 2s/step - loss: 0.3813 - accuracy: 0.8821 - val_loss:
0.3480 - val_accuracy: 0.8876
Epoch 5/50
7/7 [==============================] - 13s 2s/step - loss: 0.3596 - accuracy: 0.8821 - val_loss:
0.3475 - val_accuracy: 0.8876
Epoch 6/50
7/7 [==============================] - 13s 2s/step - loss: 0.3721 - accuracy: 0.8821 - val_loss:
0.3483 - val_accuracy: 0.8876
Epoch 7/50
7/7 [==============================] - 12s 2s/step - loss: 0.3639 - accuracy: 0.8821 - val_loss:
0.3444 - val_accuracy: 0.8876
Epoch 8/50
7/7 [==============================] - 13s 2s/step - loss: 0.3579 - accuracy: 0.8821 - val_loss:
0.3401 - val_accuracy: 0.8876
Epoch 9/50
7/7 [==============================] - 13s 2s/step - loss: 0.3587 - accuracy: 0.8821 - val_loss:
0.3370 - val_accuracy: 0.8876
Epoch 10/50
7/7 [==============================] - 13s 2s/step - loss: 0.3826 - accuracy: 0.8821 - val_loss:
0.3478 - val_accuracy: 0.8876
......
Epoch 40/50
7/7 [==============================] - 13s 2s/step - loss: 0.3453 - accuracy: 0.8821 - val_loss:
0.3142 - val_accuracy: 0.8876
Epoch 41/50
7/7 [==============================] - 13s 2s/step - loss: 0.3286 - accuracy: 0.8821 - val_loss:
0.3069 - val_accuracy: 0.8876
Epoch 42/50
7/7 [==============================] - 13s 2s/step - loss: 0.3289 - accuracy: 0.8821 - val_loss:
0.3109 - val_accuracy: 0.8876
Epoch 43/50
7/7 [==============================] - 13s 2s/step - loss: 0.3250 - accuracy: 0.8821 - val_loss:
0.3132 - val_accuracy: 0.8876
Epoch 44/50
7/7 [==============================] - 13s 2s/step - loss: 0.3273 - accuracy: 0.8821 - val_loss:
0.3153 - val_accuracy: 0.8876
Epoch 45/50
7/7 [==============================] - 13s 2s/step - loss: 0.3332 - accuracy: 0.8821 - val_loss:
0.3230 - val_accuracy: 0.8876
Epoch 46/50
7/7 [==============================] - 13s 2s/step - loss: 0.3313 - accuracy: 0.8821 - val_loss:
0.3129 - val_accuracy: 0.8876
Epoch 47/50
7/7 [==============================] - 14s 2s/step - loss: 0.3244 - accuracy: 0.8821 - val_loss:
0.3161 - val_accuracy: 0.8876
```

Epoch 48/50
7/7 [==============================] - 13s 2s/step - loss: 0.3291 - accuracy: 0.8821 - val_loss: 0.2991 - val_accuracy: 0.8876
Epoch 49/50
7/7 [==============================] - 13s 2s/step - loss: 0.3290 - accuracy: 0.8821 - val_loss: 0.2972 - val_accuracy: 0.8876
Epoch 50/50
7/7 [==============================] - 13s 2s/step - loss: 0.3273 - accuracy: 0.8821 - val_loss: 0.3031 - val_accuracy: 0.8876

附录 11

介绍：该数据是 VGG16 模型在训练时产生的部分信息

Epoch 1/50
7/7 [==============================] - 27s 4s/step - loss: 2.3581 - accuracy: 0.7264 - val_loss: 0.7164 - val_accuracy: 0.8652
Epoch 2/50
7/7 [==============================] - 26s 4s/step - loss: 0.3535 - accuracy: 0.8821 - val_loss: 0.3856 - val_accuracy: 0.8652
Epoch 3/50
7/7 [==============================] - 26s 4s/step - loss: 0.2999 - accuracy: 0.8632 - val_loss: 0.2652 - val_accuracy: 0.8989
Epoch 4/50
7/7 [==============================] - 27s 4s/step - loss: 0.1826 - accuracy: 0.9434 - val_loss: 0.1380 - val_accuracy: 0.9326
Epoch 5/50
7/7 [==============================] - 27s 4s/step - loss: 0.1483 - accuracy: 0.9528 - val_loss: 0.1691 - val_accuracy: 0.9101
Epoch 6/50
7/7 [==============================] - 27s 4s/step - loss: 0.1164 - accuracy: 0.9387 - val_loss: 0.1508 - val_accuracy: 0.9213
Epoch 7/50
7/7 [==============================] - 28s 4s/step - loss: 0.1160 - accuracy: 0.9434 - val_loss: 0.1694 - val_accuracy: 0.9438
Epoch 8/50
7/7 [==============================] - 27s 4s/step - loss: 0.1022 - accuracy: 0.9623 - val_loss: 0.1684 - val_accuracy: 0.9213
Epoch 9/50
7/7 [==============================] - 28s 4s/step - loss: 0.0993 - accuracy: 0.9623 - val_loss: 0.1856 - val_accuracy: 0.9101
Epoch 10/50
7/7 [==============================] - 28s 4s/step - loss: 0.0772 - accuracy: 0.9670 - val_loss: 0.1316 - val_accuracy: 0.9213
.....
Epoch 40/50
7/7 [==============================] - 13s 2s/step - loss: 0.0380 - accuracy: 0.9858 - val_loss:

0.1918 - val_accuracy: 0.9438
Epoch 41/50
7/7 [==============================] - 14s 2s/step - loss: 0.0302 - accuracy: 0.9953 - val_loss: 0.1050 - val_accuracy: 0.9775
Epoch 42/50
7/7 [==============================] - 13s 2s/step - loss: 0.0245 - accuracy: 0.9953 - val_loss: 0.1072 - val_accuracy: 0.9663
Epoch 43/50
7/7 [==============================] - 13s 2s/step - loss: 0.0235 - accuracy: 0.9953 - val_loss: 0.0997 - val_accuracy: 0.9663
Epoch 44/50
7/7 [==============================] - 14s 2s/step - loss: 0.0154 - accuracy: 1.0000 - val_loss: 0.1001 - val_accuracy: 0.9663
Epoch 45/50
7/7 [==============================] - 14s 2s/step - loss: 0.0241 - accuracy: 0.9906 - val_loss: 0.1208 - val_accuracy: 0.9551
Epoch 46/50
7/7 [==============================] - 14s 2s/step - loss: 0.0168 - accuracy: 1.0000 - val_loss: 0.0759 - val_accuracy: 0.9888
Epoch 47/50
7/7 [==============================] - 14s 2s/step - loss: 0.0189 - accuracy: 0.9953 - val_loss: 0.1219 - val_accuracy: 0.9551
Epoch 48/50
7/7 [==============================] - 14s 2s/step - loss: 0.0239 - accuracy: 0.9906 - val_loss: 0.1252 - val_accuracy: 0.9551
Epoch 49/50
7/7 [==============================] - 14s 2s/step - loss: - accuracy: 0.9906 - val_loss: 0.1338 - val_accuracy: 0.9551
Epoch 50/50
7/7 [==============================] - 16s 2s/step - loss: 0.0511 - accuracy: 0.9717 - val_loss: 0.1793 - val_accuracy: 0.9438

附录 12

介绍：该数据是 GoogLeNet 模型在训练时产生的部分信息

Epoch 1/50
7/7 [==============================] - 11s 1s/step - loss: 0.5816 - accuracy: 0.8443 - val_loss: 0.3400 - val_accuracy: 0.8989
Epoch 2/50
7/7 [==============================] - 8s 1s/step - loss: 0.2517 - accuracy: 0.9057 - val_loss: 0.1456 - val_accuracy: 0.9326
Epoch 3/50
7/7 [==============================] - 8s 1s/step - loss: 0.1452 - accuracy: 0.9481 - val_loss: 0.1341 - val_accuracy: 0.9663
Epoch 4/50
7/7 [==============================] - 8s 1s/step - loss: 0.1169 - accuracy: 0.9481 - val_loss:

```
0.1219 - val_accuracy: 0.9663
Epoch 5/50
7/7 [==============================] - 8s 1s/step - loss: 0.1048 - accuracy: 0.9623 - val_loss:
0.1090 - val_accuracy: 0.9888
Epoch 6/50
7/7 [==============================] - 8s 1s/step - loss: 0.0764 - accuracy: 0.9764 - val_loss:
0.1220 - val_accuracy: 0.9551
Epoch 7/50
7/7 [==============================] - 8s 1s/step - loss: 0.0930 - accuracy: 0.9670 - val_loss:
0.1548 - val_accuracy: 0.9663
Epoch 8/50
7/7 [==============================] - 8s 1s/step - loss: 0.0584 - accuracy: 0.9811 - val_loss:
0.2187 - val_accuracy: 0.9551
Epoch 9/50
7/7 [==============================] - 8s 1s/step - loss: 0.0463 - accuracy: 0.9811 - val_loss:
0.0771 - val_accuracy: 0.9663
Epoch 10/50
7/7 [==============================] - 8s 1s/step - loss: 0.0726 - accuracy: 0.9764 - val_loss:
0.1526 - val_accuracy: 0.9551
......
Epoch 40/50
7/7 [==============================] - 8s 1s/step - loss: 0.0294 - accuracy: 0.9858 - val_loss:
0.1713 - val_accuracy: 0.9551
Epoch 41/50
7/7 [==============================] - 8s 1s/step - loss: 0.0174 - accuracy: 0.9906 - val_loss:
0.1256 - val_accuracy: 0.9551
Epoch 42/50
7/7 [==============================] - 8s 1s/step - loss: 0.0086 - accuracy: 1.0000 - val_loss:
0.2999 - val_accuracy: 0.9326
Epoch 43/50
7/7 [==============================] - 8s 1s/step - loss: 0.0205 - accuracy: 0.9906 - val_loss:
0.2102 - val_accuracy: 0.9438
Epoch 44/50
7/7 [==============================] - 8s 1s/step - loss: 0.0185 - accuracy: 0.9906 - val_loss:
0.1414 - val_accuracy: 0.9663
Epoch 45/50
7/7 [==============================] - 8s 1s/step - loss: 0.0111 - accuracy: 1.0000 - val_loss:
0.0505 - val_accuracy: 0.9775
Epoch 46/50
7/7 [==============================] - 8s 1s/step - loss: 0.0205 - accuracy: 0.9906 - val_loss:
0.1836 - val_accuracy: 0.9551
Epoch 47/50
7/7 [==============================] - 8s 1s/step - loss: 0.0072 - accuracy: 1.0000 - val_loss:
0.2779 - val_accuracy: 0.9551
Epoch 48/50
7/7 [==============================] - 8s 1s/step - loss: 0.0229 - accuracy: 0.9953 - val_loss:
```

0.2093 - val_accuracy: 0.9551
Epoch 49/50
7/7 [==============================] - 8s 1s/step - loss: 0.0191 - accuracy: 0.9906 - val_loss: 0.1668 - val_accuracy: 0.9663
Epoch 50/50
7/7 [==============================] - 8s 1s/step - loss: 0.0115 - accuracy: 0.9953 - val_loss: 0.1170 - val_accuracy: 0.9663

附录 13

介绍：该数据是用 python 对图像进行大小统一处理时的输出信息

Resized skc1mm49.jpg to (224, 224)
Resized skdg8f20.jpg to (224, 224)
Resized skk9bite.jpg to (224, 224)
Resized skpmsg2e.jpg to (224, 224)
Resized skrhr1ui.jpg to (224, 224)
Resized slifadb6.jpg to (224, 224)
Resized slvybxux.jpg to (224, 224)
Resized sm4qli26.jpg to (224, 224)
Resized sm81v5fs.jpg to (224, 224)
Resized sn4arphf.jpg to (224, 224)
...
Resized zx9exkdw.jpg to (224, 224)
Resized zxfhwbki.jpg to (224, 224)
Resized zxgm3ar3.jpg to (224, 224)
Resized zy7m9c8x.jpg to (224, 224)
Resized zynixpux.jpg to (224, 224)
Resized zyylqfs0.jpg to (224, 224)
Resized zzhe6767.jpg to (224, 224)
Resized zzipq9zn.jpg to (224, 224)
Resized zzvom0ax.jpg to (224, 224)
Resized zzvth22q.jpg to (224, 224)

附录 14

介绍：该数据是用 python 输出图像特征信息的部分数据

Image 0i4iakfv:
[[3.7783108e-04 1.1336444e-02 0.0000000e+00 ... 0.0000000e+00
  6.9270271e-01 0.0000000e+00]
 [5.1893090e-04 7.5149741e-03 8.2925865e-03 ... 0.0000000e+00
  1.8814019e+00 0.0000000e+00]
 [5.8725930e-04 3.8658448e-03 3.6771819e-03 ... 2.5751984e-01
  1.2876033e+00 0.0000000e+00]
 ...
 [1.5408118e-04 3.2461833e-03 5.8066002e-03 ... 1.9579110e+00
  1.4146117e+00 0.0000000e+00]
 [0.0000000e+00 4.0194290e-03 5.8783153e-03 ... 8.4811115e-01

```
   9.2620754e-01 7.3574889e-01]
  [0.0000000e+00 1.5279163e-02 0.0000000e+00 ... 2.0853657e-01
   0.0000000e+00 1.0469673e+00]] [[3.5003162e-04 1.3811637e-02 9.2626549e-04 ... 5.7437718e-
02
   9.3266106e-01 0.0000000e+00]
  [7.3807599e-04 6.1230012e-03 3.8638809e-03 ... 0.0000000e+00
   2.2795219e+00 0.0000000e+00]
  [0.0000000e+00 4.0757458e-04 1.3345449e-02 ... 1.1137798e+00
   1.7332528e+00 0.0000000e+00]
  ...
  [5.5830495e-04 0.0000000e+00 0.0000000e+00 ... 2.8080637e+00
   2.4419765e+00 5.0000846e-02]
  [8.5575302e-05 2.8711883e-04 0.0000000e+00 ... 2.3083036e+00
   1.8685530e+00 1.4007800e+00]
  [2.0690559e-04 1.6849682e-02 0.0000000e+00 ... 1.4015732e+00
   6.9212073e-01 1.3540261e+00]] [[2.46354961e-04 3.15766595e-03 0.00000000e+00 ...
0.00000000e+00
   0.00000000e+00 6.39930964e-01]
  [5.18825720e-04 0.00000000e+00 4.70432406e-03 ... 4.83730435e-02
   6.51570916e-01 6.45007491e-02]
  [9.67239612e-04 0.00000000e+00 1.24328155e-02 ... 2.41250849e+00
   1.51153874e+00 0.00000000e+00]
  ...
  [0.00000000e+00 0.00000000e+00 0.00000000e+00 ... 4.59442711e+00
   2.83480453e+00 0.00000000e+00]
  [8.81430809e-04 0.00000000e+00 0.00000000e+00 ... 2.84964323e+00
   1.39719379e+00 6.00295782e-01]
  [1.94521097e-03 1.96916591e-02 0.00000000e+00 ... 1.56270385e+00
   8.65353703e-01 1.17301941e+00]] [[1.4768940e-04 6.9508133e-03 2.7658914e-03 ... 1.7442894e-
01
   5.1366085e-01 2.0359132e+00]
  [3.6393068e-04 0.0000000e+00 5.2024578e-03 ... 4.2977637e-01
   1.6103712e+00 1.2419209e+00]
  [4.8758640e-04 0.0000000e+00 0.0000000e+00 ... 2.7389665e+00
   2.1232331e+00 0.0000000e+00]
  ...
  [2.7569834e-04 0.0000000e+00 0.0000000e+00 ... 4.6094775e+00
   1.9363496e+00 0.0000000e+00]
  [9.9949702e-04 0.0000000e+00 0.0000000e+00 ... 3.3546429e+00
   8.9427340e-01 0.0000000e+00]
  [1.2856268e-03 1.4162829e-02 0.0000000e+00 ... 1.8711424e+00
   5.6687009e-01 0.0000000e+00]] [[4.00678779e-04 1.31463315e-02 1.63599360e-03 ...
2.20071673e-02
   8.32298160e-01 2.03498673e+00]
  [5.55600796e-04 0.00000000e+00 0.00000000e+00 ... 4.05731082e-01
   1.89832866e+00 1.46215761e+00]
```

```
  [4.11717047e-04 0.00000000e+00 0.00000000e+00 ... 2.91947722e+00
    2.71671915e+00 0.00000000e+00]
  ...
  [0.00000000e+00 4.41600941e-03 0.00000000e+00 ... 3.77006054e+00
    2.24213529e+00 0.00000000e+00]
  [5.44528128e-04 1.22017437e-03 0.00000000e+00 ... 1.85444641e+00
    1.35214972e+00 0.00000000e+00]
  [5.28100878e-04 1.51209533e-04 0.00000000e+00 ... 4.67861593e-01
    2.63426453e-01 0.00000000e+00]] [[7.3761090e-05 9.3784910e-03 0.0000000e+00 ...
4.5409685e-01
    1.2290978e+00 1.2401572e+00]
  [0.0000000e+00 0.0000000e+00 1.2543143e-03 ... 8.7192929e-01
    1.6924409e+00 1.2421720e+00]
  [0.0000000e+00 0.0000000e+00 0.0000000e+00 ... 2.9698825e+00
    2.2570286e+00 1.2273448e+00]
  ...
  [3.3376360e-04 7.4097770e-03 0.0000000e+00 ... 2.0184131e+00
    4.5795794e+00 1.3149935e-01]
  [4.5412374e-04 0.0000000e+00 0.0000000e+00 ... 0.0000000e+00
    5.3710670e+00 0.0000000e+00]
  [2.2285042e-04 0.0000000e+00 1.9778352e-04 ... 0.0000000e+00
    2.3580382e+00 0.0000000e+00]] [[0.0000000e+00 0.0000000e+00 0.0000000e+00 ... 3.5707098e-
01
    6.2834686e-01 3.2293910e-01]
  [0.0000000e+00 0.0000000e+00 0.0000000e+00 ... 1.2800550e+00
    4.7725576e-01 4.9415886e-02]
  [0.0000000e+00 0.0000000e+00 1.2244598e-04 ... 2.4688804e+00
    5.6828249e-01 1.0875895e+00]
  ...
  [2.7141476e-05 0.0000000e+00 4.3647509e-04 ... 1.2716234e+00
    3.3200836e+00 1.0314676e+00]
  [2.9489837e-04 0.0000000e+00 0.0000000e+00 ... 0.0000000e+00
    3.7723069e+00 8.3262503e-02]
  [0.0000000e+00 2.1347459e-03 0.0000000e+00 ... 0.0000000e+00
    1.2932479e+00 0.0000000e+00]]

Image 0kf0vfiw:
[[0.0000000e+00 1.0177925e-02 8.0271659e-04 ... 1.9597948e-01
    2.1533397e-01 5.9576827e-01]
  [0.0000000e+00 4.1384352e-03 0.0000000e+00 ... 0.0000000e+00
    5.3189397e-02 4.4675770e-01]
  [0.0000000e+00 0.0000000e+00 0.0000000e+00 ... 0.0000000e+00
    4.6339718e-01 0.0000000e+00]
  ...
  [0.0000000e+00 2.7080881e-03 4.0695462e-03 ... 0.0000000e+00
    4.4491714e-01 1.8293202e+00]
```

```
  [0.0000000e+00 6.1454736e-03 9.7373372e-04 ... 0.0000000e+00
    4.4672993e-01 1.3960325e+00]
  [0.0000000e+00 9.3548391e-03 0.0000000e+00 ... 0.0000000e+00
    1.2857360e-01 1.7987883e-01]] [[0.0000000e+00 8.9738537e-03 0.0000000e+00 ...
0.0000000e+00
    4.5644259e+00 3.2461813e-01]
  [2.8236999e-04 4.9823276e-03 0.0000000e+00 ... 0.0000000e+00
    5.4516106e+00 6.6817766e-01]
  [1.3793570e-04 0.0000000e+00 0.0000000e+00 ... 0.0000000e+00
    3.7317152e+00 2.3447758e-01]
  ...
  [1.9573727e-04 1.9335829e-02 3.4351316e-03 ... 0.0000000e+00
    1.4115946e+00 2.5017395e+00]
  [0.0000000e+00 6.8438076e-03 0.0000000e+00 ... 0.0000000e+00
    1.5348516e+00 1.8341286e+00]
  [0.0000000e+00 0.0000000e+00 0.0000000e+00 ... 0.0000000e+00
    1.5388141e+00 2.8053594e-01]] [[8.3766051e-04 1.2533704e-02 3.5164796e-04 ... 8.7295848e-01
    7.4941978e+00 0.0000000e+00]
  [1.0154747e-03 1.0715054e-02 0.0000000e+00 ... 5.3322244e-01
    1.0099991e+01 0.0000000e+00]
  [2.2282163e-04 0.0000000e+00 0.0000000e+00 ... 0.0000000e+00
    6.1887374e+00 0.0000000e+00]
  ...
  [7.6915603e-04 3.9748894e-03 0.0000000e+00 ... 0.0000000e+00
    9.1016328e-01 1.9987009e+00]
  [9.7072020e-04 9.7082779e-03 0.0000000e+00 ... 0.0000000e+00
    9.4392717e-01 1.9608567e+00]
  [0.0000000e+00 6.2399479e-03 0.0000000e+00 ... 0.0000000e+00
    1.2674143e+00 8.2326680e-01]] [[9.3450688e-04 3.7395118e-03 0.0000000e+00 ...
0.0000000e+00
    6.8272686e+00 0.0000000e+00]
  [0.0000000e+00 1.5254512e-02 0.0000000e+00 ... 0.0000000e+00
    1.0906713e+01 0.0000000e+00]
  [0.0000000e+00 7.3405113e-03 0.0000000e+00 ... 0.0000000e+00
    6.6859708e+00 1.9401354e-01]
  ...
  [1.3338722e-04 2.5527959e-04 1.1474937e-03 ... 0.0000000e+00
    1.0411904e+00 7.2212750e-01]
  [1.0021700e-03 0.0000000e+00 0.0000000e+00 ... 0.0000000e+00
    1.3187562e+00 3.2723978e-01]
  [8.6660712e-04 1.8717092e-03 5.5987167e-04 ... 0.0000000e+00
    1.5181587e+00 4.8210907e-01]] [[6.8450515e-04 9.6101444e-03 0.0000000e+00 ... 1.8097001e-
01
    3.5973067e+00 0.0000000e+00]
  [3.4670535e-04 2.0884199e-02 0.0000000e+00 ... 0.0000000e+00
    7.9872527e+00 0.0000000e+00]
```

```
 [0.0000000e+00 0.0000000e+00 0.0000000e+00 ... 0.0000000e+00
   5.0922027e+00 4.3571234e-02]
 ...
 [3.0074109e-04 4.4514956e-03 7.1434421e-03 ... 0.0000000e+00
   4.9197361e-01 1.3099456e+00]
 [2.9151759e-04 5.9170821e-03 0.0000000e+00 ... 0.0000000e+00
   1.1131833e+00 1.3639815e+00]
 [1.2045061e-04 0.0000000e+00 0.0000000e+00 ... 2.5519460e-01
   9.8559844e-01 1.1858783e+00]] [[5.2866346e-04 5.1815240e-03 0.0000000e+00 ...
0.0000000e+00
   1.0028732e+00 1.4370471e-01]
 [0.0000000e+00 1.8858520e-02 0.0000000e+00 ... 0.0000000e+00
   4.4036918e+00 2.3129719e-01]
 [0.0000000e+00 0.0000000e+00 0.0000000e+00 ... 0.0000000e+00
   2.3995805e+00 4.2878607e-01]
 ...
 [4.9106881e-04 8.0892136e-03 0.0000000e+00 ... 0.0000000e+00
   6.4173406e-01 2.5175915e+00]
 [0.0000000e+00 1.1903353e-02 6.8483683e-03 ... 0.0000000e+00
   1.4522500e+00 2.9463856e+00]
 [0.0000000e+00 7.6470487e-03 3.5006506e-03 ... 0.0000000e+00
   7.6350194e-01 2.6170387e+00]] [[0.0000000e+00 7.3284898e-03 0.0000000e+00 ...
0.0000000e+00
   3.6000931e-01 2.9580644e-01]
 [3.2739452e-05 1.3761522e-02 0.0000000e+00 ... 0.0000000e+00
   2.3392987e+00 5.0966954e-01]
 [7.5220232e-05 0.0000000e+00 2.6693605e-03 ... 0.0000000e+00
   1.3252456e+00 3.7421903e-01]
 ...
 [0.0000000e+00 4.0403511e-03 0.0000000e+00 ... 0.0000000e+00
   5.0204647e-01 2.1257348e+00]
 [2.2169850e-05 1.0155292e-02 0.0000000e+00 ... 0.0000000e+00
   9.9636996e-01 2.9141488e+00]
 [0.0000000e+00 2.9503668e-03 4.4891313e-03 ... 0.0000000e+00
   6.1452937e-01 2.4394307e+00]]
```

附录 15

介绍：该代码是用 python 生成的 ROC 曲线图

```python
import pandas as pd
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

# 读取 CSV 文件，将预测结果和真实标签加载到 DataFrame 中
df = pd.read_csv('E:\Acode\\ans.csv')
y_true = df['True Labels']  # 将'真实标签'替换为实际的真实标签列名
```

```
y_scores = df['Predicted Labels']  # 将'预测概率'替换为实际的预测概率列名

# 计算 ROC 曲线的假阳性率和真阳性率
fpr, tpr, thresholds = roc_curve(y_true, y_scores)

# 计算 AUC 面积
auc = roc_auc_score(y_true, y_scores)

# 绘制 ROC 曲线
plt.plot(fpr, tpr, label=f'AUC = {auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')  # 绘制对角线
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()
```